Predicting Credit-Card Default: A Comparative Study of Machine-Learning Classifiers

RUAN Hangjing
G2607546F
hangjing001@e.ntu.edu.sg

LU Xinyu
G2603537D
xinyu057@e.ntu.edu.sg

LI Jingwen
G2607526K
jingwen013@e.ntu.edu.sg

Abstract

Because a missed defaulter costs several times a false alarm, a default model must be judged at an operating point, not by ranking alone. We compare ten classifiers, including a K-nearest-neighbour benchmark and a logistic baseline, on the UCI credit-card default data (29,965 clients, 22.1% defaulters), all choices made on training rows, each configuration frozen at its first scoring on a 30% test set. Tree ensembles rank best on training CV (XGBoost: CV AUC 0.786; test 0.789, AP 0.578) but gain only 0.025 AUC over the logit, the forest is indistinguishable from XGBoost, and six months of repayment status carry nearly all the signal, additively. Class weights, SMOTE and under-sampling inside the folds move the intercept, not the ranking. At 5:1, moving the logit's cut-off from 0.5 to 0.15 saves 1,937 [1,604, 2,218] of 7,063 test cost units; switching to XGBoost saves a further 145 [−18, 299].

Contribution statement. Group B19. Model families are owned end to end (code, tuning, interpretation, presentation): RUAN Hangjing the KNN benchmark, the logistic family, nested cross-validation and PCA/K-means; LU Xinyu the pruned tree, random forest and bagging, gradient boosting and XGBoost, TreeSHAP, monotone constraints and partial dependence; LI Jingwen the support-vector machines, the neural network, LDA/QDA/naive Bayes, the class-imbalance study and leakage demonstration, Platt recalibration, and the freeze and aggregation code. Shares per section (RUAN/LU/LI, %): §1 30/20/50; §2 25/15/60; §3 30/25/45; §4 30/35/35; §5 15/60/25; §6 40/15/45; §7 34/33/33; code and reproducibility 20/25/55; presentation 33/33/34 (Table A4; red = draft awaiting the group's confirmation). Not shared with any other module.

1 Introduction

Unsecured revolving credit is one of the largest sources of loss for a retail bank: a defaulter's balance is written off, while the upside on a good customer is a thin monthly margin, so a model that merely ranks clients well is not yet a decision; the question is which model, at which cut-off.

The study is comparative in the sense of the course [6] rather than a hunt for a winner. Two models are the reference points — a K-nearest-neighbour classifier, the non-parametric benchmark, and a logistic regression, the scorecard a credit department could defend to a regulator — and every other family is read against them, each gap explained by a mechanism rather than a rank. Three methods beyond the syllabus — TreeSHAP [7], a monotonically constrained XGBoost and partial dependence [4] — carry the inference.

Objectives. Four questions organise what follows. (1) *Value of complexity*: how much ranking quality do non-linear and ensemble methods add over the two reference models, beyond test-set

noise? (2) *Operating point*: cut at 0.5, at maximum F_1, or at the threshold implied by an asymmetric cost — and does that choice outweigh the model's? (3) *Class imbalance*: do class weights, SMOTE and under-sampling inside the folds help, and what does resampling before the split fabricate? (4) *Inference*: which characteristics drive default, in which direction, and do the coefficient, impurity, permutation and Shapley accounts agree?

2 Data

Source and forward-looking design. The data are the *Default of Credit Card Clients* panel of Yeh and Lien [10], UCI dataset 350: 30,000 card holders of a Taiwanese bank. Every predictor is measured over April–September 2005 and the label is the October 2005 payment, so no feature is observed after the outcome it predicts. Twenty-three raw predictors and six engineered ones give 29 (Table A1). Code, data and every result file are public at `github.com/jingwen013-design/mh6805-credit-default`; `code/run_all.sh` regenerates every number from the public download under seed 2607526, and `code/freeze.py --check` verifies the frozen test scorings. There are 6,636 defaulters against 23,364 non-defaulters (22.1%); removing 35 exact duplicates leaves 6,630 against 23,335 at the same rate. A "never default" rule is already 77.9% accurate while catching nobody, so accuracy selects nothing here; average precision has no-skill level 0.221, the prevalence.

What happens before the split. Three things, none estimated from the data (`code/common.py`): a fixed-codebook recoding of the undocumented EDUCATION and MARRIAGE codes to "others", row-wise algebra that builds the six engineered features from a client's own columns, and the removal of 35 byte-identical rows (0.12%); everything estimated — scaling, one-hot levels, resampling, thresholds — lives inside the `Pipeline` and is re-fitted in every fold (Table A3). One imputation is self-inflicted: `PAY_RATIO_MEAN` is undefined for 936 of the 29,965 clients (3.12%) whose six bills are all non-positive and is set to 0 — a filler that is not neutral (in the training partition they default at 37.0% against 21.7%) and that merges dormant accounts with genuine non-payers.

Exploratory evidence. Arrears dominate, and capacity modulates them: in the training partition the default rate moves 12.9%, 33.9%, 67.6% from revolving credit to one and two months late in September, and falls from 32% in the lowest credit-limit quintile to 14% in the highest (Table A2). Bill size alone is worthless — the six bill amounts have univariate AUC 0.50–0.52 — but the same bills against the limit are not: utilisation is U-shaped, 24.2% default among clients with no positive balance (zero bill or credit), a trough of 14.4% at 4–13% drawn, and 28.1% in the top decile (utilisation 0.98 and above). Demographic gaps are a few points (men 24.5%, women 20.6%) beside the swing of a single month's status, and the six bills are collinear ($r = 0.80$–0.95), so any linear coefficient placed on them will be unstable (Figures A2–A6).

3 Methods

Protocol. The cleaned sample is split once, stratified on the target, into 20,975 training and 8,990 test clients under seed 2607526. Every choice — hyper-parameters, thresholds, the CV winner and the top four — is made on training rows (5-fold stratified cross-validation). Each configuration was frozen at its first test scoring: round 1, 34 variants by 18 September, before the professor's 30 September comment; RUAN Hangjing's second pass of 30 September, which re-fitted and re-scored two already-frozen configurations (KNN, plain logit) under her freeze-before-test protocol; the nine Platt-recalibrated versions on 19 October; plus the SMOTE-before-split negative control. The 46 entries are in `results/frozen_manifest.json` (Table A9), and `common.evaluate()` refuses to re-score a changed configuration. The bootstrap and cost curves read the test rows but select nothing. Gradient boosting, bagging and the monotone XGBoost are not in the proposal's method table; their configurations were fixed a priori and each was scored once in round 1 (16–18 September). TreeSHAP, partial dependence and the drop-column refits are diagnostics of the fitted models on training rows only (Tables A18–A19, Figure A21); they never read the test set. The three beyond the syllabus are the rows below the rule in Table A5.

Preprocessing and metrics. Preprocessing is part of the model: standardisation, one-hot encoding of the three categorical columns and, in the imbalance study, resampling sit inside a scikit-learn

`Pipeline` and are re-fitted on the training part of every fold. Tree models are fitted unscaled. Tuning grids, software versions and the monotone constraint vector are Tables A6–A8. Two metrics, deliberately different: hyper-parameters are chosen by cross-validated ROC-AUC, because at tuning time the operating point is unknown, and the cut-off is a later decision on training out-of-fold probabilities (maximum F_1, or minimum expected cost at an assumed 5:1 false-negative to false-positive ratio) — ranking is a property of the model, the cut-off of the lender's loss function, and accuracy serves as neither.

Did the first round flatter the cross-validation figures? The second pass answers with nested cross-validation, hyper-parameters re-chosen inside each outer fold on training rows only (Table A10; reproduced 21 October). KNN gives 0.768 ± 0.007 against the plain 5-fold 0.770, and $K = 101$ uniform is re-selected on the full training set; the logistic family gives nested AUC 0.760 for plain, ridge and lasso alike, tied within one standard error, so the 1-SE rule fixed in advance keeps the plain model, and class weighting with Platt recalibration is 0.4% *costlier* in cross-fitted 5:1 cost (12,247 against 12,195 on the training rows), so the pre-registered rule — adopt only if at least 1% cheaper — keeps the plain model. The frozen choices stand.[1]

Weighting the loss versus moving the threshold. Class-weighted XGBoost is cost-sensitive learning, and on this data it is equivalent to moving the threshold: the weighted and unweighted boosters share a test AUC of 0.7887 and differ by 0.0005 in average precision, while the F_1-optimal cut-off shifts from 0.27 to 0.55. The price is calibration, Brier 0.1311 unweighted to 0.1772 weighted, because re-weighting fits a prior that is not the portfolio's; Platt scaling [9] fitted on training out-of-fold scores restores it (0.1311). The nine weighted or resampled models were recalibrated this way, scored once on 19 October and frozen (Table A15): weight the decision, not the loss — or recalibrate.

4 Results

Reading the table. Table 1 is ordered by training CV AUC, the selection score, and grouped against the two simple models this study set out to beat; test metrics stand beside it and rank nothing. The KNN benchmark reaches CV AUC 0.770 (test 0.771), the logistic baseline 0.760 (0.764; ridge and lasso logits reproduce it to four decimals), and the whole field spans 0.05 test AUC (0.740 to 0.789) because every family re-discovers the recent repayment-status codes (`PAY_1` alone has univariate AUC 0.686); what separates them is what else they do with that signal.

What the ensembles buy. Measured against the best simple model, not the weakest, the gain is smaller than it looks: XGBoost is 0.018 test AUC above KNN, not the 0.025 above the logit. A pruned tree with 17 leaves ties the benchmark on CV (0.769 against 0.770) and beats it on test (0.776 against 0.771): the repayment signal is a step function — the root splits on the worst delay on file (`MAX_DELAY` ≤ 1.5) and the arrears branch next on the current status (`PAY_1` ≤ 1.5) — and smooth models pay for smoothing it. KNN smooths most, averaging a step over $K = 101$ neighbours in 32 scaled and one-hot dimensions (without the scaler its CV AUC falls to 0.659); the logit cannot bend at all, pricing every missed payment alike and averaging the U in utilisation into one slope (odds ratio 1.33 per SD). The forest and the two boosted ensembles turn the step into a graded score, and their margin over the single tree is concentrated in average precision (+0.041 to +0.044 AP against +0.011 to +0.013 AUC). The network clears the benchmark on CV (0.776) but stops below the ensembles: step-like delay effects cost a tree one split and a network several units.

What falls below the benchmark, and why. Three mechanisms. The RBF SVM was tuned on a 6,000-row subsample and there beat a logit fitted on the same rows (CV AUC 0.768 against 0.765),

[1] RUAN's 30 September refit of the same plain logit gave a test cost of 7,065 at 0.5 and a training out-of-fold cut-off of 0.17 costing 5,141, against 7,063 / 0.15 / 5,126 for the 16 September scoring reported throughout: two false positives apart, on a flat stretch of the plain 5-fold out-of-fold cost curve (12,054 at 0.15 against 12,064 at 0.17, 0.08%, computed from `results/logistic_p_oof.npy`; the nested cross-fit behind Table A10 is a different fold scheme, hence 0.17 there). The proposal's saving table (`results/threshold_vs_model_saving_v2.csv`) is that refit: 1,924 [1,634, 2,193] and 160 [−8, 331] at 5:1, and within a few tens of units of Table 3 elsewhere (703 against 684 at 3:1, where its cut-off is 0.26 against 0.30; 7,117 against 7,121 at 10:1); Table 3 uses the 16 September scoring.

Table 1: Headline model per family, ordered and grouped by training CV AUC (5-fold, 20,975 training rows; the selection score). Every other column is on the fixed test set ($n = 8{,}990$, prevalence 0.2212) and ranks nothing; brackets: 95% bootstrap intervals over test rows ($B = 500$; paired differences in the text). t minimises 5:1 expected cost on training out-of-fold probabilities. All 34 round-1 variants: Table A11; operating points: Table A12. †Tuned on a 6,000-row subsample (subsample CV AUC 0.768); shown is its full-training out-of-fold AUC.

Model	CV AUC	Test AUC [95% CI]	Test AP	Brier	t	Cost at 0.5	Cost at t
Reference models							
KNN ($K = 101$)	0.770	0.771 [0.758, 0.782]	0.542	0.136	0.16	6,854	5,219
Logistic regression	0.760	0.764 [0.751, 0.776]	0.523	0.138	0.15	7,063	5,126
Above the benchmark on CV							
XGBoost	0.786	0.789 [0.776, 0.800]	0.578	0.131	0.15	6,642	4,981
Gradient boosting	0.785	0.787 [0.775, 0.798]	0.576	0.131	0.14	6,630	5,060
Random forest (500 trees)	0.785	0.787 [0.774, 0.798]	0.576	0.132	0.17	6,598	4,998
MLP (64–32)	0.776	0.773 [0.760, 0.785]	0.551	0.135	0.19	6,625	5,136
Not above the benchmark on CV (the tree ties it)							
Pruned tree (17 leaves)	0.769	0.776 [0.763, 0.787]	0.535	0.133	0.16	6,933	5,085
SVM (RBF, balanced)	0.761†	0.762 [0.749, 0.774]	0.494	0.145	0.12	9,316	5,155
LDA	0.757	0.759 [0.746, 0.772]	0.525	0.139	0.12	6,618	5,181
Gaussian naive Bayes	0.736	0.740 [0.726, 0.753]	0.504	0.203	0.25	5,422	5,337

but refitted on all 20,975 rows its out-of-fold AUC is 0.761: the tuning set was not the fitting set, and hinge loss is not a proper scoring rule, so ordering away from the margin is not what it optimises. LDA shares the logit's linear form (cosine 0.93 between the coefficient vectors) but estimates it from a pooled Gaussian covariance on bill and payment columns with skewness 2.7–32.6, so tail noise enters the discriminant; QDA, with a covariance per class, lets it in twice. Naive Bayes is last because independence fails where the data are richest: the six bill columns correlate at 0.80–0.95, so one balance is counted six times, and 35.8% of its test probabilities land below 0.01 or above 0.99.

Which differences are real. In a paired bootstrap over the 8,990 test rows ($B = 500$), XGBoost's margin over the KNN benchmark, +0.0181 [+0.0125, +0.0237] AUC, excludes zero, as do its margins over the logit (+0.0249 [+0.0183, +0.0314]), the network and the pruned tree. Inside the ensemble group it does not: the random forest sits 0.0020 [−0.0010, +0.0051] behind, so the two are indistinguishable and we claim no winner between them; gradient boosting is borderline (+0.0015 [+0.0001, +0.0030] on AUC, nothing on AP); XGBoost's own interval is wider than every gap within that group. Calibration: for the ensembles and the logit, Brier 0.131 (XGBoost), 0.131 (gradient boosting), 0.132 (forest) and 0.138 (logit), with XGBoost's mean probability 0.220 against prevalence 0.221, so the outputs can be read as PDs (Figure A11); naive Bayes (0.203) and QDA (0.166) cannot, their assumption failures pushing probability mass to the extremes.

Imbalance moves the threshold, not the ranking. With the samplers inside the folds [2], class weighting, SMOTE and 1:1 under-sampling move test AUC by at most 0.0012 for the logit and 0.0076 for the forest (0.7744–0.7820), never upwards by more than noise (on CV the weighted logit is +0.0007 above the plain one). What moves is the intercept ($+1.24$ log-odds for the weighted logit): balanced-at-0.5 agrees with plain-at-0.22 on 99.4% of test rows, so the apparent recall gain, 0.32 to 0.63, is a relabelled threshold bought with calibration (Brier 0.138 to 0.190), which Platt scaling on training scores buys back (0.139, 5:1 cut-off 0.15; Table A15). As a negative control we ran SMOTE on all 29,965 rows *before* the split: the forest's "test" AUC reaches 0.914, +0.140 over the honest 0.774, because 72.6% of the leaky test defaulters are synthetic points interpolated from training neighbours, while the logit moves only to 0.770 — a leak inflates *and* re-orders, and cross-validation cannot catch it once it precedes the split.

Segmentation, kept secondary. RUAN Hangjing's 30 September configuration (reproduced 21 October; Tables A16–A17, Figure A41) runs PCA on the 32 preprocessed columns (10 components carry 81.8% of the variance) and K-means on them. $K = 5$ has the highest silhouette, 0.269 (weak to moderate; $K = 2$–8: 0.260/0.267/0.257/0.269/0.209/0.215/0.218), is stable over 100 bootstrap refits (ARI 0.96 [0.75, 0.99]; per-cluster Jaccard 0.97/0.97/0.79/0.97/0.95), and Ward linkage agrees

with it only moderately (ARI 0.45–0.59). As an extra predictor the one-hot label changes the logit's AUC by $+0.0001$ [-0.0004, $+0.0007$] and AP by -0.0003 [-0.0016, $+0.0010$] in 5×5 repeated CV on the training set with the Nadeau–Bengio interval [8], so by the rule fixed in advance it is not used and the test set is not consulted for it. Cluster default rates range from 6.9% to 58.5% only because the inputs contain the delay and limit variables — the label as a score has AUC 0.648 against 0.724 for months in arrears alone — so the segments are descriptive, not economically distinct.

5 Interpretation

Why these data suit tree-based methods. Three properties of a credit file, not a taste for complexity, explain the ensembles' edge. First, arrears are ordinal with a threshold, not a slope, so a tree splits where collections changes treatment while one coefficient must price every missed payment alike. Second, exposure is U-shaped — the dormant client and the client at the limit both default more than one drawing 4–13% of the line — and the partial-dependence curve keeps that shape (interior minimum 0.204 at 0.084 utilisation, 0.024 below the left end and 0.040 below the right; Figure A21). Third, the columns mix NT$ amounts, ordinal codes and demographics, which a scale-free splitter takes untransformed. Yet the mechanism buys little — the margins of Table 1 are hundredths of AUC — because thresholds and a shallow U are small departures from additivity. The proposal expected arrears to interact with limit and utilisation; they do not: Friedman's H for the September status against the credit limit is 0.061 on the probability scale, and the largest departure from the sum of the one-way curves is 0.011 against a surface range of 0.254 (Figure A22). The effects add, and the trees win at the margins.

Four ways to rank a variable, and what they disagree about. All four measures in Table 2 put the September status first; agreement stops there. Across the 29 predictors, Spearman between impurity importance (MDI) and Breiman's out-of-bag permutation [1] is 0.310 ($n = 29$); the two permutation measures agree at 0.719, and mean $|\text{SHAP}|$ agrees with them at 0.756 and 0.772 but with MDI only at 0.433. MDI fails in the textbook direction, rewarding columns with many split points: the August and July codes rank 4th and 5th on MDI but 22nd and 24th out of bag. Permutation says how much *this fitted model* leans on a column, not what would be lost without it; refitting answers the second: dropping mean utilisation costs the forest nothing (-0.00004 out-of-bag AUC), dropping the September status 0.0058 (Table A19). Only the current arrears state is irreplaceable.

Direction, not just magnitude. A Shapley value [7] is a feature's average marginal contribution over all orders in which features could enter, so contributions sum exactly to the gap between a client's log-odds and the average client's (the base value, -1.263 log-odds, is the portfolio rate 0.2205). Importance cannot say which way a variable pushes; a signed attribution can: arrears push towards default (correlation between a client's value and its own contribution $+0.69$ for the September code, $+0.92$ for the longest delay, $+0.96$ for the delay count), a larger limit away (-0.81; Figure A17) — the signs a credit officer would state before seeing the model.

Table 2: Four ways to rank the same predictor (ranks over all 29). MDI, in-sample permutation and out-of-bag permutation come from the tuned random forest, mean $|\text{SHAP}|$ from the tuned XGBoost; "OOB AUC drop" is the out-of-bag AUC lost when the column is permuted in the *fitted* forest. Full table, rank correlations and refit losses: Tables A18–A19.

	Random forest, rank			OOB AUC	XGBoost
Credit-risk reading (variable)	MDI	Train	OOB	drop	$\lvert\text{SHAP}\rvert$
Arrears in September (PAY_1)	1	1	1	0.0114	1
Worst delay, six months (MAX_DELAY)	2	7	7	0.0009	2
Months in arrears (N_MONTHS_DELAYED)	3	17	19	0.0000	3
Average utilisation (UTIL_MEAN)	6	2	2	0.0021	5
Latest balance (BILL_AMT1)	12	5	3	0.0018	4
Credit limit (LIMIT_BAL)	13	4	4	0.0017	6
Arrears in August (PAY_2)	4	22	22	0.0000	21
Arrears in July (PAY_3)	5	23	24	-0.0001	24

Table 3: Threshold saving versus model saving at six cost ratios r (FN : FP), test set $n = 8{,}990$; all nine ratios in Table A14. Threshold saving: the frozen logit moved from 0.5 to its training out-of-fold cut-off $t_{\log}$; model saving: the further reduction from XGBoost at its own training cut-off; "Flagged": share of the test book the logit flags at $t_{\log}$; brackets: 95% paired-bootstrap intervals over test rows ($B = 1{,}000$).

r	$1/(1+r)$	$t_{\log}$	Logit cost at 0.5	Threshold saving [95% CI]	Model saving [95% CI]	Flagged
1	0.500	0.50	1,659	0	87 [44, 131]	10.5%
2	0.333	0.33	3,010	238 [141, 338]	123 [65, 185]	22.2%
3	0.250	0.30	4,361	684 [541, 830]	127 [47, 214]	24.3%
5	0.167	0.15	7,063	1,937 [1,604, 2,218]	145 [−18, 299]	44.4%
10	0.091	0.10	13,818	7,121 [6,395, 7,791]	420 [187, 646]	79.9%
20	0.048	0.01	27,328	20,335 [18,909, 21,621]	150 [−52, 333]	99.9%

A model that reads like a scorecard. Right signs on average are weaker than a lender needs. On the median profile the unconstrained booster moves against the required direction on 24 grid steps across the 13 constrained features (3 on the September status), and swept on 200 random profiles the September status lowers its risk on all 200 (576 of 2,000 steps): a client told another missed payment improved their score. Constraining XGBoost ($+1$ on the repayment codes, delay counters, utilisation and balance trend; -1 on limit and payment ratio; Table A8) removes every violation (Figure A23), for test AUC 0.789 to 0.787 (-0.0016; paired bootstrap mean -0.0017, $[-0.0030, -0.0002]$) and AP 0.578 to 0.573. A bank would pay: the loss is a seventh of the sampling noise on one test AUC (± 0.012), and a score that falls as arrears rise is indefensible to a regulator.

6 Choosing the operating point

A credit model is deployed as a cut-off, and 0.5 is optimal only when both errors cost the same. On the test set the logit at 0.5 is 81.5% accurate (majority rule 77.9%) yet recovers only 32% of defaulters, and at 5:1 it costs 7,063 units against 7,001 for flagging *every* customer (the F_1-optimal cut-offs, 0.25–0.42, and their costs: Table A12).

Why 5:1, and why a range. The ratio is an assumption with anchors: 5:1 is the Statlog German Credit cost matrix [5]; our own training rows imply more — a defaulter's median bill is NT$19,868, so at 50–70% loss given default a missed default destroys NT$10–14k, while a false alarm forgoes about NT$340 a month of margin (1.5% of the median good bill, NT$22,915) for 1–3 months, an implied 10:1 to 40:1 — so 5:1 is one point of a 1:1–20:1 sweep (Table 3; every ratio and model: Table A13).

The cut-off is worth more than the model. At 5:1 (Table 3), moving the logit's cut-off from 0.5 to the training out-of-fold optimum 0.15 saves 1,937 [1,604, 2,218] of its 7,063 test cost units (27%); replacing it by XGBoost at its own cut-off saves a further 145 [−18, 299], an interval that includes zero. At every ratio above 1:1 the threshold saving grows with the ratio while the model saving stays between 123 and 450 units, its interval including zero at 5:1 and again at 20:1 — where the exercise degenerates: the cut-off is 0.01, flagging 99.9% of the book, the trivial flag-everyone rule. The ratio is an operational dial, not a model-selection criterion: the two indistinguishable ensembles are 17 units apart at 5:1 and 20 the other way at 10:1 (Table A13), within noise.

Theory and search agree — if the scores are probabilities. Minimising expected cost gives the closed form $t = C_{FP}/(C_{FN} + C_{FP}) = 1/(1+r)$, 0.167 at 5:1; the search over training out-of-fold probabilities returns 0.15 for both the logit and XGBoost (Tables 1 and 3). The agreement is a calibration test passed (XGBoost: Brier 0.131); naive Bayes, Brier 0.203, fails it: its searched cut-off is 0.25, and 0.15 costs more (5,473 against 5,337 on the frozen test scores).

Different objectives point to different models. XGBoost, the CV winner, serves ranking (test AUC 0.789, AP 0.578; at its training cut-off 0.15 its test cost is 4,981, recall 0.784, 25% below its 6,642 at 0.5); the monotone XGBoost serves governance, where nine cost units (4,990 at 0.14,

0.2%) buy a score that can never fall when a borrower delays another month; and the 17-leaf tree serves where every rule must be readable to a customer or an auditor (CV 0.769, test 0.776).

7 Conclusion and limitations

(1) Complexity. XGBoost [3] reaches test AUC 0.789 / AP 0.578 against 0.764 / 0.523 for the logistic scorecard and 0.771 for the KNN benchmark; the field spans 0.05 AUC, the ensembles are tied (XGBoost over the forest, 0.0020 $[-0.0010, +0.0051]$) and nested cross-validation confirms the frozen choices: boosting gains only what step effects in arrears are worth.

(2) Operating point. At every cost ratio from 2:1 to 20:1 the threshold saving exceeds the model saving (at 5:1, 1,937 against 145 of the logit's 7,063 test cost units; Table 3), and the model saving's interval includes zero at 5:1 and 20:1; the operating point, not the model, is the lever.

(3) Imbalance. Nothing improves ranking: weights, SMOTE and under-sampling inside the folds leave the logit within 0.0012 AUC and cost the forest 0.0076; re-weighting moves the intercept ($+1.24$), buying recall with calibration (Brier 0.138 to 0.190) that Platt scaling restores (0.139). Resampling before the split fabricates 0.914.

(4) Inference. Recent and repeated arrears dominate — mean |SHAP| 0.298, 0.281 and 0.230 log-odds for PAY_1, MAX_DELAY and N_MONTHS_DELAYED, all pushing towards default — and SHAP tracks out-of-bag permutation (Spearman 0.77) while impurity importance tracks neither (0.31 and 0.43). Friedman's $H = 0.061$ for PAY_1 $\times$ LIMIT_BAL refutes the proposal's interaction hypothesis; monotone constraints cost 0.0016 AUC and remove every violation; the K-means segments add nothing to the logit and are descriptive only.

Limitations. One split of one 2005 Taiwanese portfolio, and a bootstrap that prices test-row noise only. Six months of history caps what any model can learn, the likeliest reason no family passes 0.79 AUC. SHAP and permutation describe the fitted model, not causation, and the imputed zero bucket of §2 conflates dormant accounts with genuine non-payers. The test set was read by 45 frozen scorings (46 manifest entries, one the leaky negative control) and by the reporting-only bootstrap and cost curves: a report card, not a fresh holdout.

Future work. Repeated splits and a fresh holdout for honest intervals, with an indicator separating the imputed zero bucket from genuine non-payment. Bureau, income and transaction data over a longer window, measured loss-given-default in place of 5:1, and isotonic recalibration.

References

[1] Breiman, L. (2001) Random forests. *Machine Learning* **45**(1):5–32.

[2] Chawla, N. V., Bowyer, K. W., Hall, L. O. & Kegelmeyer, W. P. (2002) SMOTE: synthetic minority over-sampling technique. *J. Artificial Intelligence Research* **16**:321–357.

[3] Chen, T. & Guestrin, C. (2016) XGBoost: a scalable tree boosting system. *Proc. KDD*, pp. 785–794.

[4] Friedman, J. H. (2001) Greedy function approximation: a gradient boosting machine. *Annals of Statistics* **29**(5):1189–1232.

[5] Hofmann, H. (1994) *Statlog (German Credit Data)*. UCI Machine Learning Repository, dataset 144.

[6] James, G., Witten, D., Hastie, T. & Tibshirani, R. (2021) *An Introduction to Statistical Learning*, 2nd ed. Springer.

[7] Lundberg, S. M. & Lee, S.-I. (2017) A unified approach to interpreting model predictions. *Advances in Neural Information Processing Systems* **30**:4765–4774.

[8] Nadeau, C. & Bengio, Y. (2003) Inference for the generalization error. *Machine Learning* **52**(3):239–281.

[9] Platt, J. C. (1999) Probabilistic outputs for support vector machines and comparisons to regularized likelihood methods. In *Advances in Large Margin Classifiers*, MIT Press, pp. 61–74.

[10] Yeh, I.-C. & Lien, C.-H. (2009) The comparisons of data mining techniques for the predictive accuracy of probability of default of credit card clients. *Expert Systems with Applications* **36**(2):2473–2480.

A Appendix

Everything in this appendix is generated from `results/` by `code/make_report_appendix.py`, so the tables cannot drift from the artefacts submitted with the code. The course guideline exempts appendices from the seven-page limit on the body.

Table A1: Variables in the cleaned data set: 29 predictors and the target, 29,965 clients after removing 35 exact duplicate rows.

Group	Variables	Type	Meaning
Credit line	`LIMIT_BAL`	numeric	credit limit granted (NT$), individual plus family
Demographics	`SEX`, `EDUCATION`, `MARRIAGE`, `AGE`	3 categorical, 1 numeric	gender; graduate / university / high school / others; married / single / others; age in years
Repayment status	`PAY_1`, …, `PAY_6`	ordinal ($-2 \ldots 8$)	monthly status Sep $\rightarrow$ Apr 2005: -2 no consumption, -1 paid in full, 0 revolving credit, $k \geq 1$ = k months of delay
Bill statements	`BILL_AMT1`, …, `BILL_AMT6`	numeric	monthly bill amount Sep $\rightarrow$ Apr (NT$); may be negative
Payments	`PAY_AMT1`, …, `PAY_AMT6`	numeric	amount paid in each month Sep $\rightarrow$ Apr (NT$)
Engineered	`UTIL_1`, `UTIL_MEAN`, `PAY_RATIO_MEAN`, `N_MONTHS_DELAYED`, `MAX_DELAY`, `BILL_TREND`	numeric	utilisation of the limit (latest, mean); mean payment-to-bill ratio; months in arrears; worst delay; six-month balance trend relative to the limit
Target	`DEFAULT`	binary	1 = default on the October 2005 payment (22.1% in training)

Table A2: Default rate along the two strongest credit-risk gradients, training partition only ($n = 20,975$; overall rate 22.1%). Source: `results/eda_summary.json`.

September status	Clients	Default	Credit-limit quintile	Clients	Default
No consumption (-2)	1,904	13.7%	NT$10k–50k	5,429	32.0%
Paid in full (-1)	3,941	16.9%	NT$50k–100k	3,370	25.5%
Revolving credit (0)	10,380	12.9%	NT$100k–180k	4,224	19.5%
1 month late	2,540	33.9%	NT$180k–270k	3,810	17.1%
2 months late	1,871	67.6%	NT$270k–1M	4,142	13.7%
3 months late	234	77.3%			

Table A3: Data-quality checks run on the raw download before modelling, and what was done about each finding. Reproduced verbatim from `results/eda_data_quality.csv`; the three cleaning steps that precede the split (duplicate removal, the fixed recoding of undocumented codes, the row-wise engineered features) estimate nothing from the data, and everything else is left to the in-fold pipeline.

Check	Finding	Action
Missing values	0 cells (all 24 columns complete)	None needed — no imputation step in the pipeline
Exact duplicate rows	35 of 30,000 (0.12%)	Dropped in common.clean() → 29,965 rows
Data types	All 24 columns integer; no strings, no dates	—
EDUCATION undocumented codes 0/5/6	0: 14, 5: 280, 6: 51 (total 345, 1.15%)	Recoded to 4 = others/unknown
MARRIAGE undocumented code 0	54 rows (0.18%)	Recoded to 3 = others
PAY_1..6 undocumented codes -2 and 0	PAY_1: -2 in 2,759 rows, 0 in 14,737 rows (codebook lists only -1 and 1..9)	Kept as ordinal: -2 = no consumption, -1 = paid in full, 0 = revolving credit / minimum paid, k>0 = k months late
Negative bill amounts	3,932 cells in 1,930 rows (6.4%); min BILL_AMT = -339,603	Kept — a negative bill is a credit balance (overpayment / refund)
Zero payments	PAY_AMT = 0 in at least one month for 14,542 rows (48.5%); all six months zero for 1,432 rows (4.8%)	Kept — 'no payment' is informative; PAY_RATIO_MEAN handles zero bills
Zero bills in all six months	866 of the raw 30,000 rows (2.9%); 936 of the 29,965 cleaned rows (3.12%) have all six bills NON-POSITIVE	Kept — inactive or credit-balance accounts. For the non-positive group PAY_RATIO_MEAN is undefined and imputed 0; in the training partition that group defaults at 37.0% against 21.7% for the rest, so the imputed value is informative rather than neutral
Bill above credit limit (UTIL_1 > 1)	2,115 rows (7.1%); max UTIL_1 = 6.46	Kept — over-limit usage is a genuine risk signal
LIMIT_BAL range	10,000 – 1,000,000 NT$ (median 140,000)	Plausible; heavy right tail → StandardScaler inside the pipeline, no trimming
AGE range	21 – 79 years (median 34)	Plausible; no outlier removal
Class imbalance	6,630 defaults of 29,965 = 22.1%	Kept as is; addressed downstream by class_weight / threshold tuning; AUC & AP as metrics
Train / test split	20,975 / 8,990 rows, stratified 70/30, seed 2607526; default rate 22.13% / 22.12%	Fixed once in data/processed; test set untouched in this EDA

Table A4: Contribution by section. Percentages are of the writing and analysis behind each section; the model families below are owned end to end — data, tuning, interpretation and presentation — by the member named. The percentages are the group's draft of 21 October 2026 and are typeset in red until all three members have confirmed them.

Section	RUAN	LU	LI	Basis
Abstract, §1 Introduction	30%	20%	50%	LI drafted; RUAN and LU revised
§2 Data	25%	15%	60%	LI wrote the cleaning and split contract and the exploratory analysis
§3 Methods	30%	25%	45%	shared protocol; RUAN the nested cross-validation, LI the freeze and recalibration protocol
§4 Results	30%	35%	35%	RUAN: KNN, logistic family, second pass; LU: tree, forest, GBM, XGBoost; LI: SVM, MLP, LDA/QDA/NB, imbalance study, aggregation and bootstrap
§5 Interpretation	15%	60%	25%	LU owns TreeSHAP, the monotone constraints, partial dependence and the forest importances
§6 Choosing the operating point	40%	15%	45%	RUAN: cost-ratio table and nested CV; LI: recalibration and threshold analysis
§7 Conclusion and limitations	34%	33%	33%	drafted jointly
Code and reproducibility	20%	25%	55%	LI: `common.py`, aggregation, freeze, recalibration
Presentation	33%	33%	34%	three presenters, 10 minutes

Each model family is run end to end by its owner: **RUAN Hangjing** — KNN benchmark; logistic family (plain, ridge, lasso, class-weighted); PCA and K-means; **LU Xinyu** — Pruned tree; random forest and bagging; gradient boosting and XGBoost; TreeSHAP; monotone-constrained XGBoost; partial dependence; **LI Jingwen** — Support-vector machines; feed-forward neural network; LDA, QDA and naive Bayes; class-imbalance study and the leakage demonstration. The owner of every file in `results/` is recorded in its `member` field, so this table is checkable against the submitted code. This project is not shared with any other module.

Table A5: Model families, the lecture that introduces each one, and what each is in the study to test. The last three rows are beyond the syllabus.

Family (lecture)	What it is there to test
PCA, K-means, hierarchical clustering (B2–B3)	Whether default risk lies along the directions of largest variance, and whether a segment label carries anything a supervised model has not already found.
K-nearest neighbours (B4)	Benchmark: how much of the signal a scaled distance and a local default rate recover with no model of the risk at all.
Class weights, SMOTE, under-sampling (B4)	Whether re-balancing changes the ordering of clients or only the prior behind it; and what resampling before the split fabricates.
Logistic regression: plain, ridge, lasso (B5–B6)	Whether the ceiling on this data is variance, which shrinkage can cut, or the bias of a model linear in the features, which it cannot.
LDA, QDA, naive Bayes (B6)	The price of distributional assumptions: a shared covariance, class-specific covariances on skewed amounts, and independence among near-collinear balances.
Decision tree, cost-complexity pruning (B7)	The bias–variance trade-off made readable: delay thresholds and their interactions, bought at the cost of very few distinct scores.
Random forest and bagging (B7)	Whether de-correlating trees, not merely averaging them, is what removes variance; out-of-bag error and permutation importance come free.
Gradient boosting and XGBoost (B7)	Whether sequential bias reduction beats variance reduction on a noisy target, and how much of that depends on shrinkage.
Support-vector machines, linear and RBF (B8)	Whether a margin loss that ignores ordering beyond the margin can rank risk as well as a proper scoring rule.
Feed-forward neural network (B9)	Whether learned interactions help when the strongest predictors are ordinal delay codes with step-like effects.
TreeSHAP (beyond the syllabus)	Direction and size of each variable's push on each individual client, rather than one global ranking; additive by construction, so it can be checked.
Monotonically constrained XGBoost (beyond the syllabus)	The cost, in ranking quality, of the scorecard requirement that more arrears may never lower predicted risk.
Partial dependence and ICE (beyond the syllabus)	Whether the fitted surface carries the U-shaped utilisation effect a linear term cannot express, and whether effects add or interact.

Table A6: Hyper-parameter grids searched for the headline models and the setting chosen, read from `results/<slug>.json`. Every grid is scored by 5-fold stratified cross-validation on the training partition (seed 2607526); the test set plays no part. The last column is the training-set selection score of the chosen setting (CV AUC; out-of-fold AUC for the subsample-tuned RBF SVM).

Model	Grid searched	Chosen	Sel. AUC
XGBoost	`max_depth` {3, 4, 6} × `learning_rate` {0.03, 0.1} × `n_estimators` {300, 600} × `min_child_weight` {5, 1} (24 settings); `subsample` = 0.8, `colsample_bytree` = 0.8 fixed; rounds then checked on a learning curve to 1500	`max_depth` = 3, `learning_rate` = 0.03, `n_estimators` = 300, `min_child_weight` = 5	0.7863
Gradient boosting (sklearn)	`learning_rate` {0.03, 0.1} × `max_depth` {3, 2, 4} × `n_estimators` {200, 500} (12 settings) on a 10,487-row training subsample; staged CV curve on the full training set	`learning_rate` = 0.03, `max_depth` = 3, `n_estimators` = 200, `subsample` = 0.8	0.7850
Random forest	`max_features` {sqrt, 0.3, 0.5} × `min_samples_leaf` {20, 5, 1} (9 settings) at 200 trees; top two re-run at 500 trees	`max_features` = sqrt, `min_samples_leaf` = 20, `n_estimators` = 500	0.7849
MLP (feed-forward NN)	`hidden_layer_sizes` {32, 64, 64x32, 128x64} × `alpha` {0.0001, 0.001, 0.01} × `learning_rate_init` {0.001, 0.003} (24 settings); `relu`, `adam`, early stopping fixed	`hidden_layer_sizes` = 64x32, `alpha` = 0.001, `learning_rate_init` = 0.001	0.7762
KNN	`K` {5 … 151} (12 values) × `weights` {uniform, distance} (24 settings)	`n_neighbors` = 101, `weights` = uniform	0.7700
Decision tree (pruned)	`ccp_alpha` {0 … 0.02493} (21 values) (cost-complexity path) × `max_depth` {None, 4, 6, 8}	`criterion` = entropy, `ccp_alpha` = 0.000663, `max_depth` = 6	0.7689
SVM (RBF kernel)	`C` {0.1, 1, 10} × `gamma` {scale, 0.01, 0.1} × `class_weight` {None, balanced} on a 6,000-row training subsample; one-standard-error rule	`C` = 1, `gamma` = 0.01, `class_weight` = balanced; Platt-scaled	0.7606
Logistic regression (plain)	none: unpenalised reference fit	`C` = inf (unpenalised), `solver` = lbfgs	0.7602
Logistic regression (L1 lasso, C tuned)	`C` {0.001 … 100} (11 values)	`C` = 100, `penalty` = l1 (l1_ratio=1)	0.7602
LDA (linear discriminant analysis)	`shrinkage` {None, auto, 0.05, 0.2, 0.5}	`solver` = lsqr, `shrinkage` = None	0.7572
Gaussian naive Bayes	`var_smoothing` {1e-09 … 0.1} (9 values)	`var_smoothing` = 0.01	0.7356

Table A7: Software versions and the single seed behind every result in this report. Versions are the ones recorded in the result files by the scripts that produced them.

Component	Version / value
Python	3.13.5
scikit-learn	1.9.0
XGBoost	3.4.1
SHAP	0.52.0
NumPy	2.5.1
pandas	3.0.5
imbalanced-learn	0.14.2
SciPy	1.18.0
Random seed (split, folds, samplers, every model)	2607526
Cross-validation	5 stratified folds of the training partition, shuffled with the seed
Split	stratified 70/30, 20,975 training / 8,990 test rows

Table A8: The monotone constraint vector of the constrained XGBoost, fixed a priori from credit logic and read back from the fitted booster: `(-1,0,1,1,1,1,1,1,0,0,0,0,0,0,0,0,0,0,0,1,1,-1,1,1,1,0,0,0,0,0,0)` over the 32 preprocessed columns (one-hot categories count once per raw variable here). Hyper-parameters are inherited unchanged from the tuned unconstrained model.

Sign	Variables	n
$+1$ (more may only raise the predicted risk)	`PAY_1`, `PAY_2`, `PAY_3`, `PAY_4`, `PAY_5`, `PAY_6`, `N_MONTHS_DELAYED`, `MAX_DELAY`, `UTIL_1`, `UTIL_MEAN`, `BILL_TREND`	11
-1 (more may only lower it)	`LIMIT_BAL`, `PAY_RATIO_MEAN`	2
0 (unconstrained)	`AGE`, `BILL_AMT1`, `BILL_AMT2`, `BILL_AMT3`, `BILL_AMT4`, `BILL_AMT5`, `BILL_AMT6`, `PAY_AMT1`, `PAY_AMT2`, `PAY_AMT3`, `PAY_AMT4`, `PAY_AMT5`, `PAY_AMT6`, `SEX`, `EDUCATION`, `MARRIAGE`	19

Sweeping each constrained variable with the others at their training median, the unconstrained booster moves against the required direction on 24 grid steps in total (3 of them on `PAY_1`); over 200 random training profiles it violates monotonicity in `PAY_1` on 576 of 2,000 steps (largest fall 0.036); the constrained booster on none. Price of the constraint: CV AUC -0.0019, test AUC -0.0016 (paired bootstrap, $B = 500$: mean -0.0017, 95 % interval $[-0.0030, -0.0002]$).

Table A9: Freeze timeline: the 46 entries of `results/frozen_manifest.json` grouped by round. A configuration is frozen the first time it is scored on the test set, and `common.evaluate()` refuses to score a changed configuration under a frozen slug (`code/freeze.py --check` verifies every stored prediction vector).

Round	Scored	n	What	Selection basis
1	16 Sep–18 Sep	34	every model variant of Table A11: each configuration tuned on training CV, then scored on the test set once and frozen (before the professor's 30 September comment); none re-scored since	5-fold CV AUC on the training partition; both thresholds on training out-of-fold probabilities
2	30 Sep	2	RUAN Hangjing's second pass under her freeze-before-test protocol (`frozen_choices_ruan_v2.json`): the KNN ($K = 101$, uniform) and plain-logistic configurations re-fitted and re-scored; adopted on 21 October unchanged; the headline tables keep the round-1 scoring (predictions differ by $\leq 10^{-2}$, test AUC by 5×10^{-5})	nested CV with the pre-registered 1-SE and 1 % cost rules (Table A10); training rows only
3	19 Oct	9	Platt-recalibrated versions of the nine class-weighted or resampled models (Table A15), each scored once	Platt map fitted on training out-of-fold scores; thresholds re-chosen on cross-fitted training scores
—	17 Sep	1	negative control: SMOTE applied before the split (the leakage demonstration); its 'test' rows are not the real test set and it is not a model result	none — deliberately leaky

Table A10: Nested cross-validation of the KNN and logistic families, RUAN Hangjing's second pass of 30 September 2026 (`logistic_nested_cv_v2.csv`, `knn_v2.json`). The 21 October reproduction on the training rows (`nested_cv_summary_repro.json`, `logistic_nested_cv_repro.csv`) matched the nested AUCs and standard errors, the logistic rows and the selected configurations; the KNN Brier and calibration-slope columns are RUAN's 30 September out-of-fold figures (`knn_v2.json`), which the reproduction (`nested_cv_knn_repro.json`) puts at 0.1395 and 0.966. The pre-registered rules re-select the configurations frozen on 16 September.

Model	Nested AUC	Nested AP	Brier	Cal. slope	5:1 cut-off	5:1 cost
Logistic, plain (unpenalised)	0.7602 ± 0.0050	0.5049	0.1410	0.982	0.17	12,195
Logistic, ridge (C tuned inside each fold)	0.7601 ± 0.0051	0.5049	0.1410	0.985	0.17	12,205
Logistic, lasso (C tuned inside each fold)	0.7602 ± 0.0050	0.5049	0.1410	0.982	0.15	12,188
Logistic, class-weighted	0.7608 ± 0.0055	0.5020	0.1920	0.974	0.39	12,263
Logistic, class-weighted + Platt	0.7608 ± 0.0055	0.5022	0.1412	1.004	0.15	12,247
KNN (K, weights tuned in-fold; $K = 101$ uniform chosen)	0.7684 ± 0.0073	—	0.1392	0.989	—	—

Nested CV: outer loop = the project's five folds, inner loop = a 5-fold grid search inside each outer training fold, so the AUC is that of the whole tuning procedure. Columns: nested AUC $\pm$ its standard error over the outer folds (for KNN, the standard deviation), nested average precision, Brier score and calibration slope of the nested out-of-fold scores, the 5:1 cut-off chosen on those scores (closed form $1/6 = 0.167$) and the cross-fitted 5:1 cost on the 20,975 training rows. Pre-registered rule: 1-SE on nested AUC among plain > ridge > lasso; adopt Platt-calibrated class weighting only if its cross-fitted 5:1 cost is > 1% lower. Outcome: plain kept; the Platt-calibrated weighting is 0.43 % costlier (12,247 against 12,195), so the rule (adopt only if $> 1\,\%$ cheaper) is not met and it is not adopted. For comparison, the plain 5-fold out-of-fold 5:1 cost of the frozen 16 September logit (`results/logistic_p_oof.npy` against the 20,975 training labels) is 12,054 at 0.15 and 12,064 at 0.17 (+0.08 %): a different cross-fit from the nested one, which is why the two costs at 0.17 differ. The test set is not read anywhere in this table.

Table A11: Every model variant fitted in this project (34 in total), ordered by CV AUC, the training-set selection score (for the two SVMs, tuned on a subsample, the out-of-fold AUC on the full training partition). CV AUC is the mean 5-fold out-of-fold AUC on the training partition; the remaining columns are on the held-out test set, reported once and not used to rank. RUAN Hangjing's 30 September re-scoring of the KNN and plain-logistic configurations (Table A9) is not a separate model and is not listed.

Model	CV AUC	Test AUC	Test AP	Brier
XGBoost	0.786	0.789	0.578	0.131
XGBoost (class-weighted)	0.786	0.789	0.579	0.177
Gradient boosting (sklearn)	0.785	0.787	0.576	0.131
Random forest	0.785	0.787	0.576	0.132
XGBoost (monotone constraints)	0.784	0.787	0.573	0.132
Bagging (RF, max_features=None)	0.781	0.783	0.571	0.133
Random forest (500 trees) - class_weight='balanced'	0.781	0.782	0.569	0.161
Random forest (500 trees) - none	0.780	0.782	0.569	0.133
Random forest (500 trees) - under-sampling in folds	0.779	0.781	0.558	0.185
MLP (feed-forward NN)	0.776	0.773	0.551	0.135
Random forest (500 trees) - SMOTE in folds	0.774	0.774	0.554	0.153
MLP (tuned, balanced sample weights)	0.774	0.771	0.535	0.185
KNN	0.770	0.771	0.542	0.136
Decision tree (pruned)	0.769	0.776	0.535	0.133
LinearSVC + Platt calibration	0.761	0.763	0.515	0.139
Logistic regression (plain, class_weight=balanced)	0.761	0.763	0.517	0.190
Logistic (C=1) - class_weight='balanced'	0.761	0.763	0.517	0.190
SVM (RBF kernel)†	0.761	0.762	0.494	0.145
Logistic regression (L2 ridge, C tuned)	0.760	0.764	0.523	0.138
Logistic regression (plain)	0.760	0.764	0.523	0.138
Logistic regression (L1 lasso, C tuned)	0.760	0.764	0.523	0.138
Logistic (C=1) - none	0.760	0.764	0.523	0.138
Logistic (C=1) - SMOTE in folds	0.760	0.762	0.516	0.191
Logistic + K-means segment dummies (round-1 scoring; not adopted by the pre-registered rule)	0.760	0.763	0.522	0.138
Logistic (C=1) - under-sampling in folds	0.760	0.763	0.515	0.190
LDA (linear discriminant analysis)	0.757	0.759	0.525	0.139
Logistic regression on principal components only	0.755	0.762	0.518	0.139
Gaussian naive Bayes (log1p amounts)	0.754	0.756	0.508	0.193
QDA (quadratic discriminant analysis)	0.751	0.755	0.516	0.166
Gaussian naive Bayes	0.736	0.740	0.504	0.203
SVM (linear kernel)†	0.717	0.751	0.506	0.148
KNN (K=5)	0.706	0.709	0.421	0.161
MLP over-parameterised (256,256)	0.692	0.678	0.400	0.212
Decision tree (unpruned full tree)	0.615	0.613	0.290	0.272

†Tuned on a 6,000-row training subsample; the CV AUC shown is the out-of-fold AUC of the chosen setting refitted on the full training partition (the same figure as Table 1). The CV AUC on the tuning subsample, not comparable with the other rows, was SVM (RBF kernel) 0.768; SVM (linear kernel) 0.763.

Table A12: Operating points of the 11 headline configurations of `aggregate_summary.json` (ten model families: the lasso logit duplicates the plain logit to four decimals and is omitted from Table 1). Both thresholds are chosen on training out-of-fold probabilities (maximum F_1; minimum 5:1 cost); recall is at the cost threshold; the cost columns are the expected test cost at a 5:1 false-negative to false-positive ratio.

Model	F_1 thr.	Cost thr.	Recall	Cost @0.5	Cost @F_1	Cost @cost thr.
KNN	0.25	0.16	0.75	6,854	5,224	5,219
Logistic regression (plain)	0.30	0.15	0.74	7,063	5,417	5,126
Logistic regression (L1 lasso, C tuned)	0.30	0.15	0.74	7,064	5,417	5,125
LDA (linear discriminant analysis)	0.27	0.12	0.70	6,618	5,352	5,181
Gaussian naive Bayes	0.41	0.25	0.64	5,422	5,385	5,337
Decision tree (pruned)	0.28	0.16	0.76	6,933	5,308	5,085
Random forest	0.31	0.17	0.76	6,598	5,293	4,998
Gradient boosting (sklearn)	0.26	0.14	0.80	6,630	5,186	5,060
XGBoost	0.27	0.15	0.78	6,642	5,164	4,981
SVM (RBF kernel)	0.42	0.12	0.69	9,316	5,355	5,155
MLP (feed-forward NN)	0.27	0.19	0.71	6,625	5,282	5,136

Trivial rules for reference: flagging every client costs 7,001, flagging none costs 9,945.

Table A13: Cost-ratio sensitivity: expected test cost of each model at its own cost-minimising threshold, as the false-negative to false-positive ratio varies.

Model	1:1	2:1	3:1	5:1	8:1	10:1	20:1
Gradient boosting	1,566	2,617	3,562	5,060	5,910	6,278	6,856
KNN	1,638	2,720	3,662	5,219	6,221	6,525	6,898
Logistic regression	1,659	2,772	3,677	5,126	6,381	6,697	6,976
MLP	1,594	2,711	3,664	5,136	6,121	6,468	6,998
Pruned tree	1,573	2,685	3,582	5,085	6,027	6,359	7,001
Random forest	1,573	2,671	3,559	4,998	5,992	6,257	6,850
SVM (RBF)	1,684	2,726	3,637	5,155	6,364	6,702	7,088
XGBoost	1,572	2,649	3,550	4,981	5,892	6,277	6,843

Table A14: Threshold saving against model saving at nine false-negative to false-positive cost ratios r. Threshold saving: expected test cost of the frozen plain logistic at the 0.5 cut-off minus its cost at the cut-off chosen on training out-of-fold scores. Model saving: the logistic at its own cut-off minus the frozen XGBoost at its own cut-off. Brackets are 95 % paired-bootstrap intervals. Source: `results/threshold_vs_model_saving.csv` (Table 3 in the body is the six-ratio excerpt).

r	$1/(1+r)$	Logit cut-off	XGB cut-off	Logit cost @0.5	Threshold saving	Model saving
1:1	0.500	0.50	0.48	1,659	0 [0, 0]	87 [44, 131]
2:1	0.333	0.33	0.32	3,010	238 [141, 338]	123 [65, 185]
3:1	0.250	0.30	0.27	4,361	684 [541, 830]	127 [47, 214]
4:1	0.200	0.19	0.19	5,712	1,228 [1,003, 1,450]	167 [46, 287]
5:1	0.167	0.15	0.15	7,063	1,937 [1,604, 2,218]	145 [−18, 299]
7:1	0.125	0.12	0.13	9,765	3,682 [3,186, 4,146]	442 [218, 649]
10:1	0.091	0.10	0.09	13,818	7,121 [6,395, 7,791]	420 [187, 646]
15:1	0.062	0.06	0.06	20,573	13,507 [12,415, 14,473]	450 [243, 652]
20:1[†]	0.048	0.01	0.05	27,328	20,335 [18,909, 21,621]	150 [−52, 333]

Paired bootstrap of the 8,990 test rows, $B = 1000$, seed 2607526; the test set is read for these intervals but selects nothing (both cut-offs are chosen on training out-of-fold scores; the closed form is $1/(1+r)$). The model-saving interval includes zero at 5:1, 20:1. [†]At 20:1 the logistic cut-off flags 99.9 % of clients, i.e. the trivial flag-everyone rule, so the threshold 'saving' there is not a model result. Proposal-stage table (RUAN Hangjing's 30 September refit of the same plain logit, cut-off 0.17): threshold saving 1,924 [1,634, 2,193], model saving 160 [−8, 331] at 5:1.

Table A15: Platt recalibration of the nine class-weighted or resampled models. The Platt map is fitted on each model's training out-of-fold scores; the 5:1 cut-off is re-chosen on cross-fitted training scores; each recalibrated version was then scored on the test set once (19 October) and frozen. Columns: mean predicted probability on test (prevalence 0.221); test Brier score, with the unweighted twin in brackets; 5:1 cut-off; test cost at the closed-form threshold 1/6.

Model	Mean p	Brier (twin)	5:1 cut-off	Cost at $1/6$
Logistic, class-weighted	0.440 → 0.221	0.1901 → 0.1389 (0.1383)	0.39 → 0.15	6,828 → 5,093
Logistic (C=1), class-weighted	0.440 → 0.221	0.1901 → 0.1389 (0.1383)	0.41 → 0.15	6,829 → 5,086
Logistic (C=1), SMOTE in folds	0.440 → 0.222	0.1909 → 0.1391 (0.1383)	0.41 → 0.18	6,732 → 5,138
Logistic (C=1), under-sampled	0.441 → 0.221	0.1904 → 0.1390 (0.1383)	0.41 → 0.15	6,831 → 5,086
Forest, class-weighted	0.371 → 0.220	0.1606 → 0.1335 (0.1332)	0.36 → 0.18	5,860 → 5,023
Forest, SMOTE in folds	0.339 → 0.222	0.1529 → 0.1359 (0.1332)	0.32 → 0.18	5,679 → 5,124
Forest, under-sampled	0.432 → 0.222	0.1850 → 0.1345 (0.1332)	0.44 → 0.18	6,201 → 5,020
XGBoost, class-weighted	0.423 → 0.220	0.1772 → 0.1311 (0.1311)	0.37 → 0.14	6,251 → 4,950
MLP, balanced sample weights	0.433 → 0.227	0.1855 → 0.1377 (0.1353)	0.42 → 0.19	6,608 → 5,088

Table A16: Choice of K for K-means on the first 10 principal components of the 32 preprocessed columns (81.8 % of the variance; 2, 5 and 15 components carry 48, 65 and 92 %), training partition only. RUAN Hangjing's analysis of 30 September 2026 (`kmeans_selection_v2.csv`), reproduced on 21 October (`kmeans_selection_repro.csv`, identical). Bold: the K with the highest silhouette, chosen by the rule fixed in advance.

K	Silhouette	Bootstrap ARI (mean)	ARI 5th pct.	Min. cluster Jaccard
2	0.260	0.986	0.971	0.993
3	0.267	0.987	0.976	0.980
4	0.257	0.976	0.939	0.974
5	**0.269**	0.941	0.745	0.768
6	0.209	0.787	0.544	0.670
7	0.215	0.957	0.905	0.889
8	0.218	0.954	0.863	0.404

Two bootstrap runs, not one. The three right-hand columns come from the selection sweep over K (for $K = 5$: ARI 0.941, minimum cluster Jaccard 0.768). The stability figures quoted in the body and in Table A17 come from the separate 100-refit stability run of the chosen $K = 5$ (`unsupervised_summary_v2.json`): ARI 0.957 [0.745, 0.990], per-cluster Jaccard 0.972/0.974/0.791/0.970/0.953; agreement with Ward's hierarchical clustering at the same K: ARI 0.45–0.59 over five bootstrap samples (moderate).

Table A17: Profiles of the 5 K-means segments on the training partition (RUAN Hangjing, 30 September 2026, `kmeans_profiles_v2.csv`; reproduced 21 October). Jaccard: per-cluster stability over the bootstrap refits. The default rates differ because the inputs contain the delay and limit variables; the segments are descriptive, not economically distinct.

Cluster	n	Share	Default rate	Mean limit (NT$)	Months delayed	Utilisation	Jaccard
0	7,977	38.0%	16.8%	214,213	0.30	0.04	0.972
1	2,619	12.5%	58.5%	79,798	4.32	0.58	0.974
2	886	4.2%	6.9%	324,323	0.15	0.22	0.791
3	7,560	36.0%	17.9%	98,695	0.35	0.57	0.970
4	1,933	9.2%	18.2%	285,867	0.53	0.75	0.953

Incremental value of the one-hot segment label as an extra predictor of the plain logit, 5×5 repeated CV on the training rows with the Nadeau–Bengio corrected interval: ΔAUC $+0.0001$ $[-0.0004, +0.0007]$, ΔAP -0.0003 $[-0.0016, +0.0010]$; adoption rule (both lower bounds > 0) not met, adopted = False, so the test set was not consulted for it. The label used as a score alone has AUC 0.648 against 0.724 for months-in-arrears alone; the logit on the 10 PCs alone changes AUC by -0.034 against the logit on the raw columns.

Table A18: Four ways to rank the same 29 predictors: mean |SHAP| on the tuned XGBoost over the 20,975 training clients, and the random forest's impurity decrease, training-set permutation and out-of-bag permutation importances. Ranks, not magnitudes, are comparable across columns.

Feature	mean |SHAP|	rank	MDI	rank	perm. (train)	rank	perm. (OOB)	rank
PAY_1	0.2980	1	0.1876	1	0.0423	1	0.0114	1
MAX_DELAY	0.2815	2	0.1018	2	0.0145	7	0.0009	7
N_MONTHS_DELAYED	0.2303	3	0.0935	3	0.0097	17	0.0000	19
BILL_AMT1	0.1128	4	0.0274	12	0.0158	5	0.0018	3
UTIL_MEAN	0.1100	5	0.0368	6	0.0187	2	0.0021	2
LIMIT_BAL	0.1024	6	0.0264	13	0.0163	4	0.0017	4
PAY_AMT1	0.0858	7	0.0299	9	0.0158	6	0.0007	8
UTIL_1	0.0709	8	0.0313	7	0.0185	3	0.0010	6
BILL_TREND	0.0709	9	0.0292	10	0.0134	9	0.0004	10
PAY_AMT3	0.0667	10	0.0233	17	0.0123	10	0.0002	15
PAY_AMT2	0.0646	11	0.0240	15	0.0139	8	0.0011	5
MARRIAGE	0.0600	12	0.0045	29	0.0034	26	0.0002	14
BILL_AMT5	0.0484	13	0.0222	18	0.0108	14	−0.0000	23
PAY_AMT4	0.0441	14	0.0212	19	0.0119	11	0.0007	9
SEX	0.0436	15	0.0048	28	0.0035	25	0.0004	11
EDUCATION	0.0319	16	0.0054	27	0.0041	24	0.0001	18
AGE	0.0220	17	0.0191	22	0.0078	21	0.0003	12
BILL_AMT3	0.0207	18	0.0205	20	0.0108	15	0.0002	16
PAY_AMT5	0.0190	19	0.0178	25	0.0093	19	−0.0007	28
PAY_AMT6	0.0189	20	0.0182	24	0.0102	16	0.0001	17
PAY_2	0.0168	21	0.0558	4	0.0064	22	0.0000	22
PAY_RATIO_MEAN	0.0165	22	0.0303	8	0.0118	12	−0.0001	25
BILL_AMT4	0.0160	23	0.0201	21	0.0095	18	−0.0003	27
PAY_3	0.0124	24	0.0376	5	0.0047	23	−0.0001	24
BILL_AMT2	0.0106	25	0.0236	16	0.0117	13	0.0002	13
BILL_AMT6	0.0089	26	0.0184	23	0.0081	20	−0.0008	29
PAY_5	0.0053	27	0.0243	14	0.0028	28	0.0000	20
PAY_4	0.0033	28	0.0277	11	0.0033	27	−0.0002	26
PAY_6	0.0025	29	0.0175	26	0.0027	29	0.0000	21

Spearman rank correlation over the 29 predictors (`random_forest.json` `extra.spearman`, `shap_summary.json` `spearman_vs_other_measures`): MDI vs out-of-bag permutation 0.310, MDI vs training-set permutation 0.538, training-set vs out-of-bag permutation 0.719; mean |SHAP| vs MDI 0.433, vs training-set permutation 0.756, vs out-of-bag permutation 0.772.

Table A19: Refitting the tuned random forest without one column, for the six columns with the largest out-of-bag permutation drop. Permutation measures how much the fitted forest leans on a column; refitting measures what is lost if the column is never available. Out-of-bag AUC on the training rows; the test set is not used.

Column removed	OOB AUC	OOB AUC lost	OOB perm. drop
(none: full model)	0.7849	0.00000	—
PAY_1	0.7791	0.00577	0.0114
UTIL_MEAN	0.7849	−0.00004	0.0021
BILL_AMT1	0.7841	0.00076	0.0018
LIMIT_BAL	0.7846	0.00028	0.0017
PAY_AMT2	0.7839	0.00096	0.0011
UTIL_1	0.7849	−0.00002	0.0010

A.1 Exploratory analysis

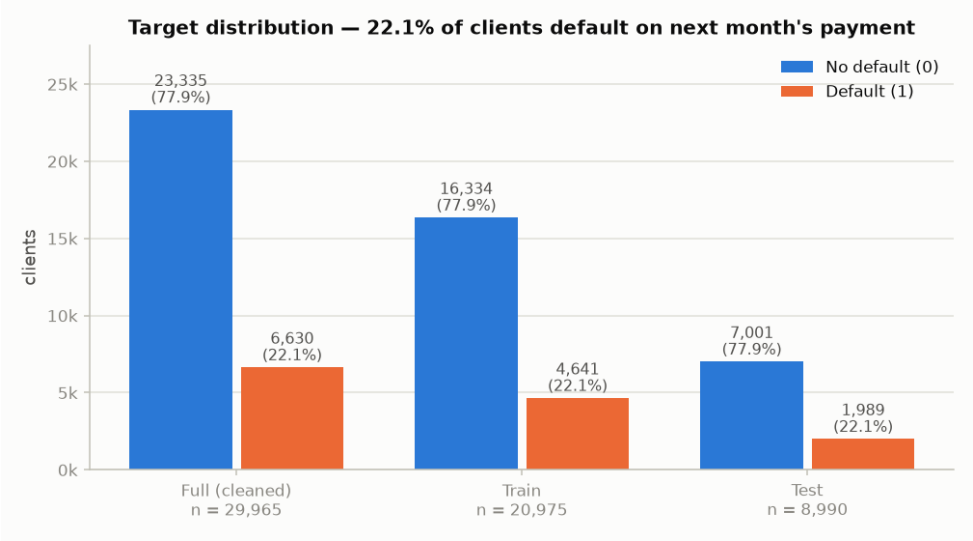


Figure A1: Class balance of the cleaned sample and of the stratified training and test partitions.

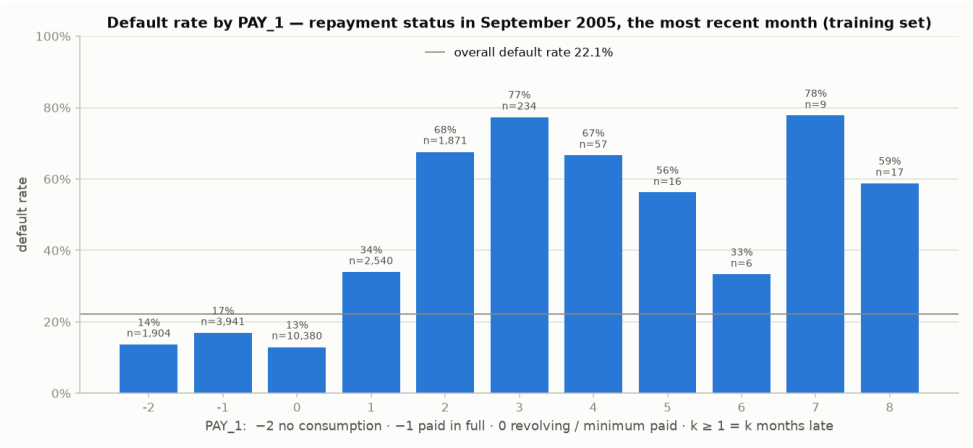


Figure A2: Default rate by the most recent repayment status, training partition.



Figure A3: Training-partition default rate by credit-limit quintile, age band, education, marriage and sex.

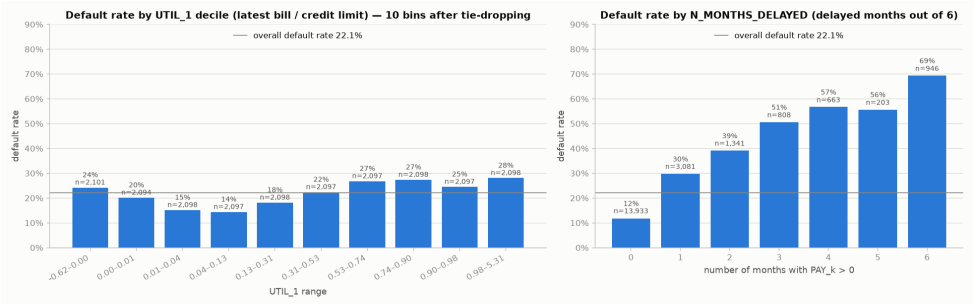


Figure A4: Default rate by utilisation decile and by number of months in arrears, training partition.

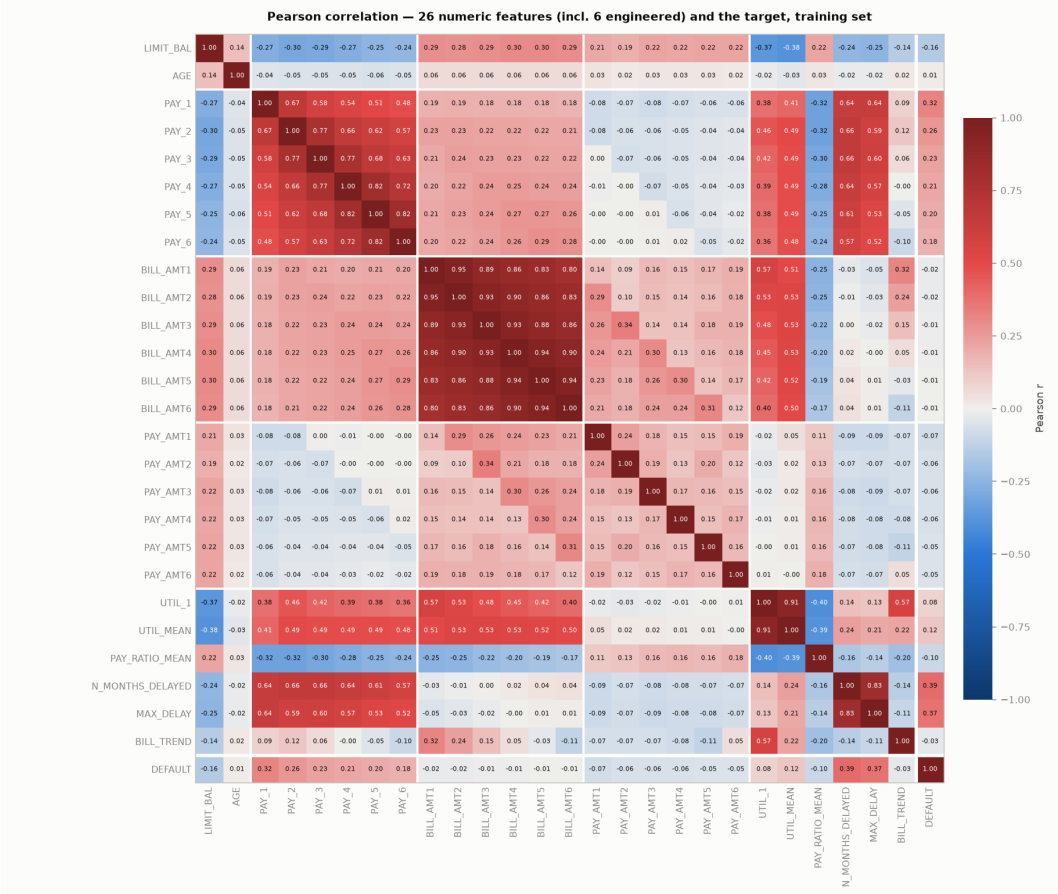


Figure A5: Correlation heat-map of the 29 predictors and the target.

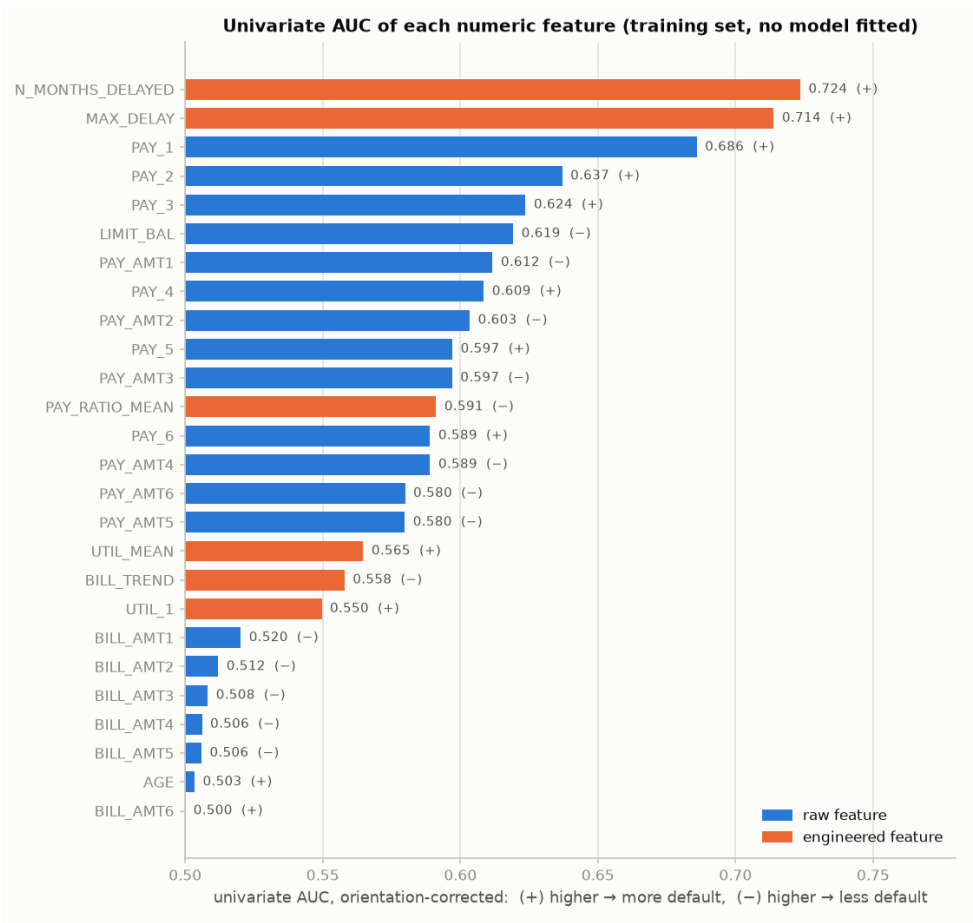


Figure A6: Univariate strength of every predictor, as an oriented AUC.

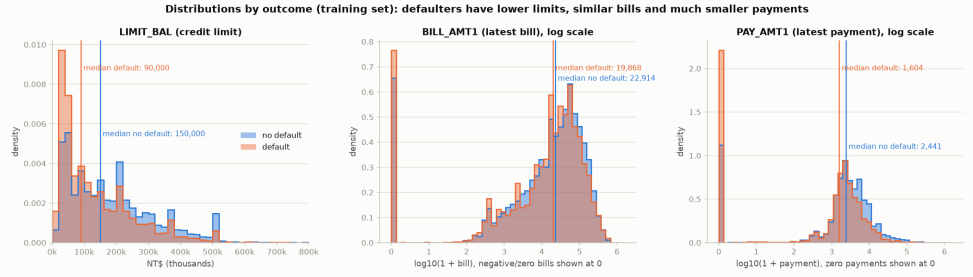


Figure A7: Distributions of the main numeric features by class, training partition.

A.2 Model comparison

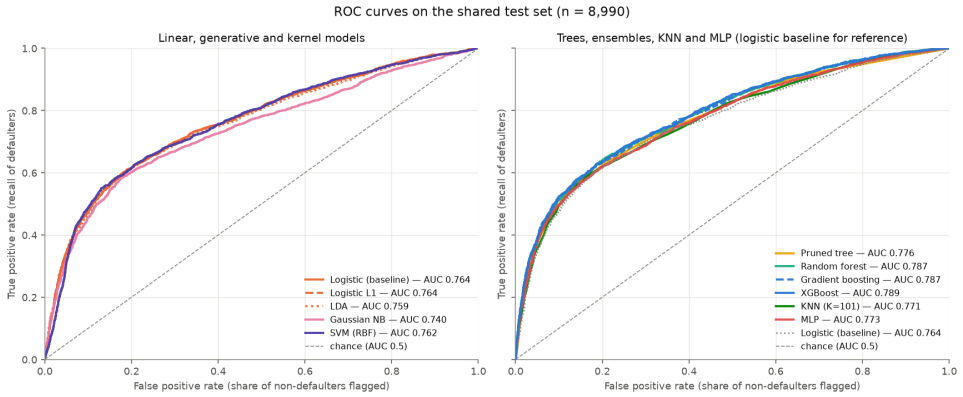


Figure A8: Test-set ROC curves of the headline models.

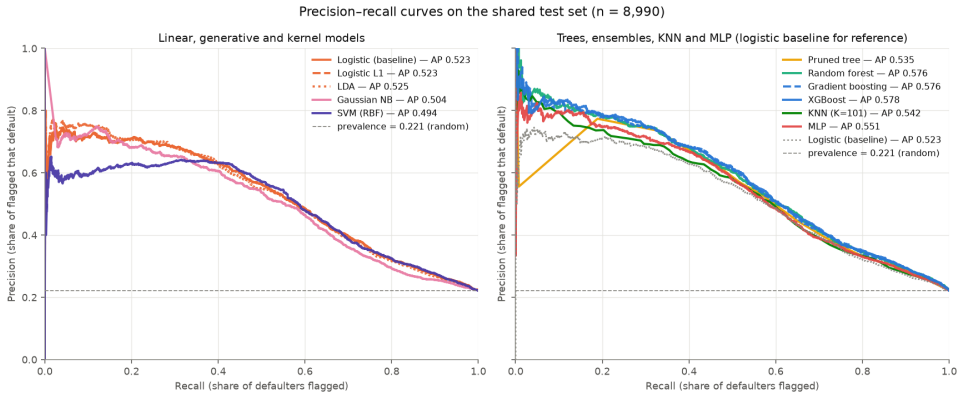


Figure A9: Test-set precision–recall curves; the horizontal line is the prevalence.

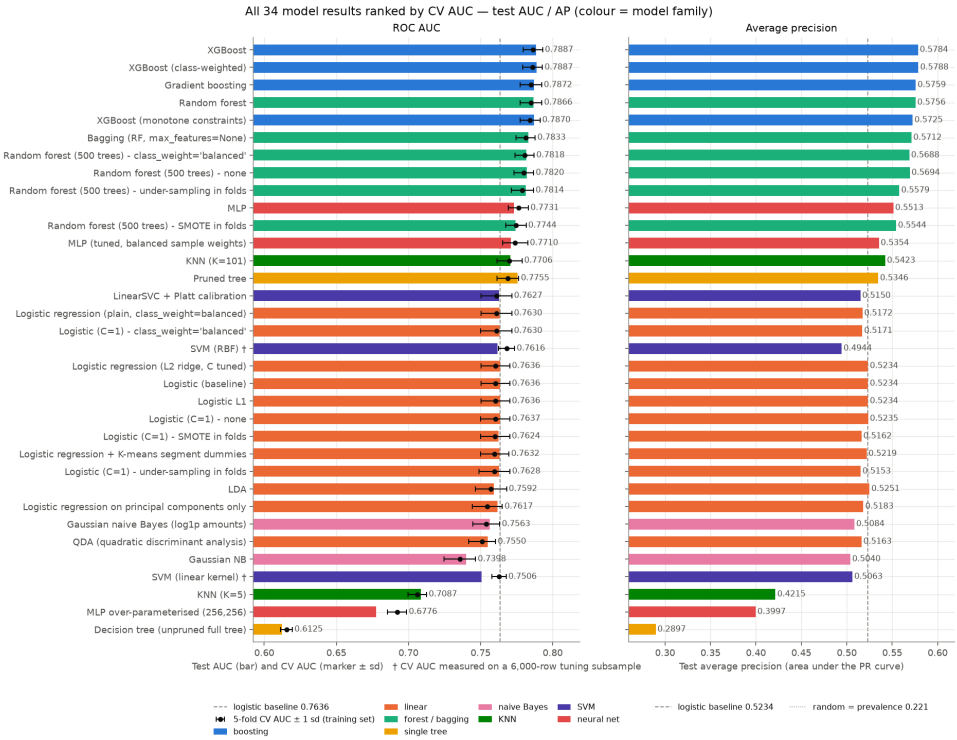


Figure A10: Test AUC of every model variant fitted, ordered by CV AUC, with the benchmark marked.

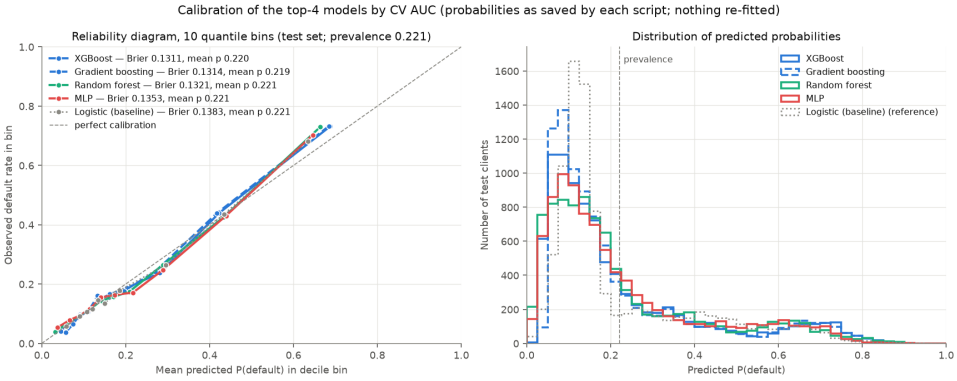


Figure A11: Calibration curves of the four models with the highest CV AUC: predicted against observed default rate by decile of score.

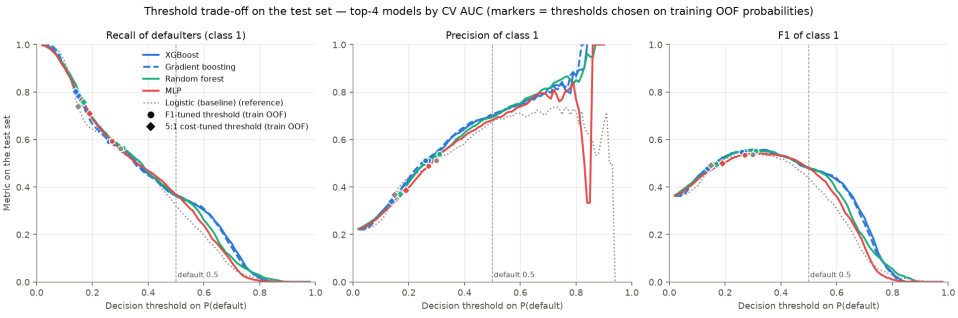


Figure A12: Recall, precision and F1 against the decision threshold.

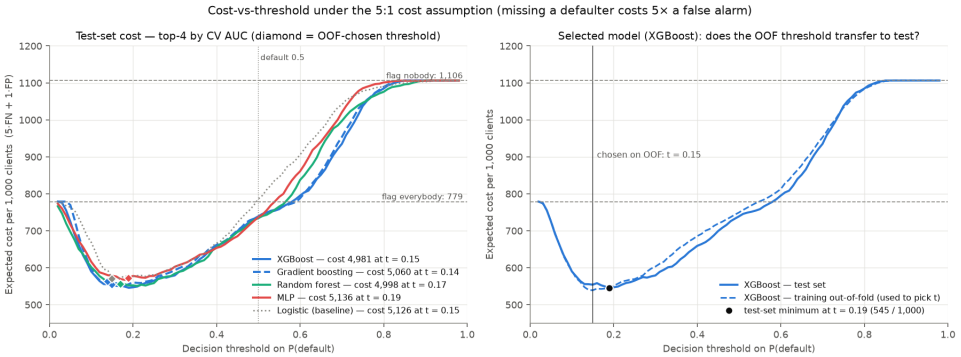


Figure A13: Expected cost against the decision threshold at a 5:1 ratio.

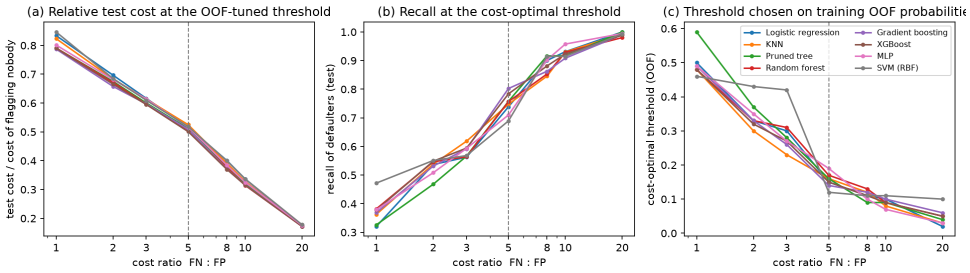


Figure A14: Cost-minimising threshold and cost as the cost ratio varies from 1:1 to 20:1.

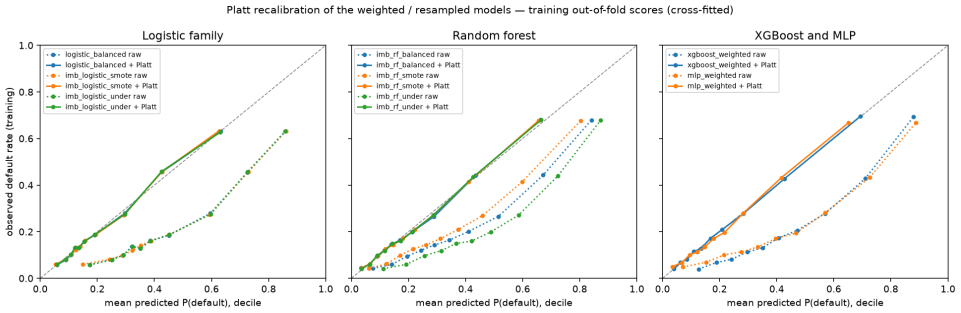


Figure A15: Reliability of the nine weighted or resampled models before and after Platt recalibration, on training out-of-fold scores.

A.3 Interpretation

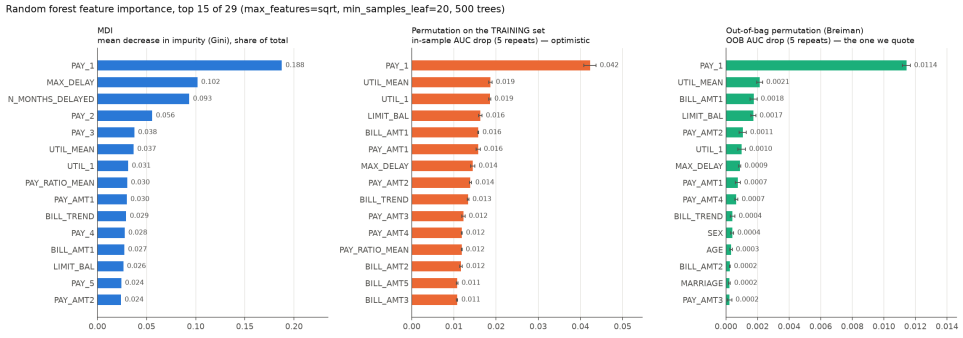


Figure A16: Random forest importances: impurity decrease, training-set permutation and out-of-bag permutation, side by side.

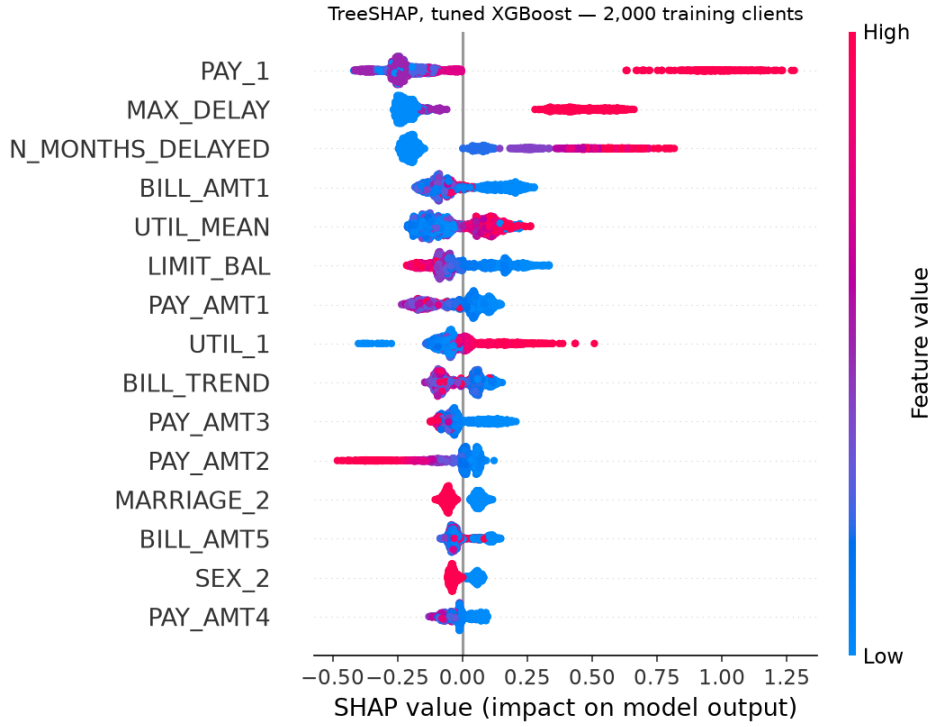


Figure A17: TreeSHAP beeswarm for the tuned XGBoost over training clients.

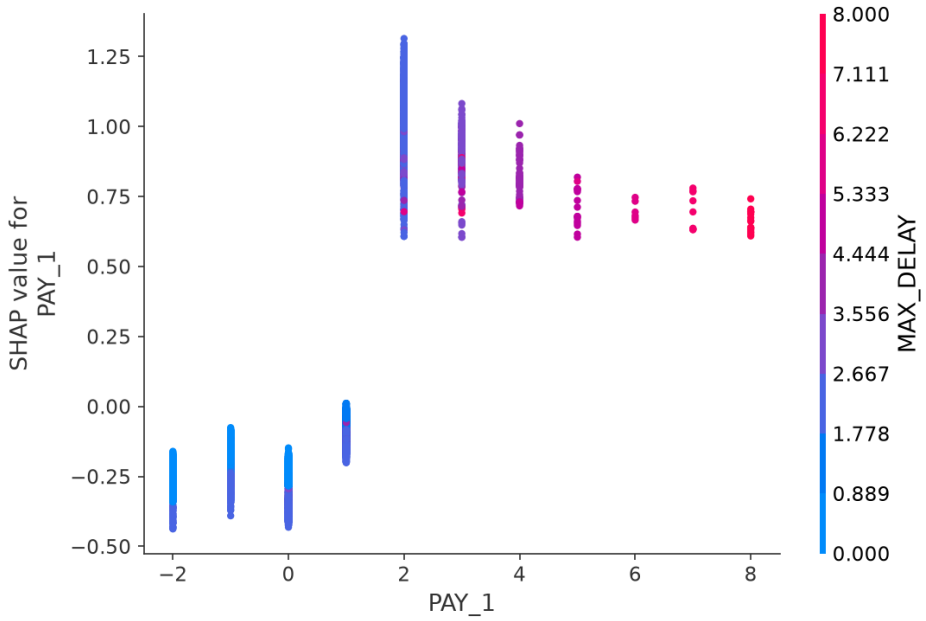


Figure A18: TreeSHAP dependence plot for the most recent repayment status.

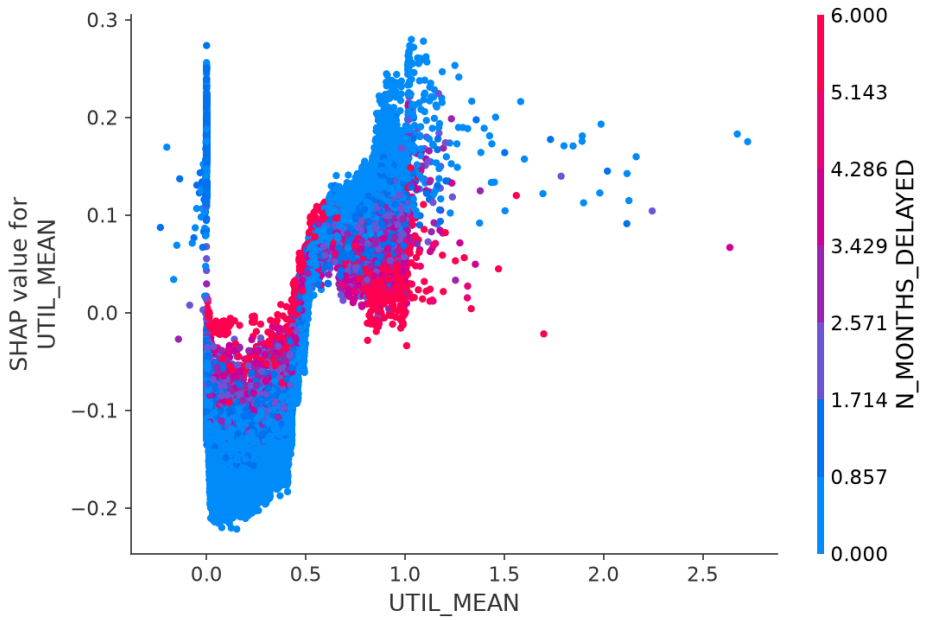


Figure A19: TreeSHAP dependence plot for mean utilisation.

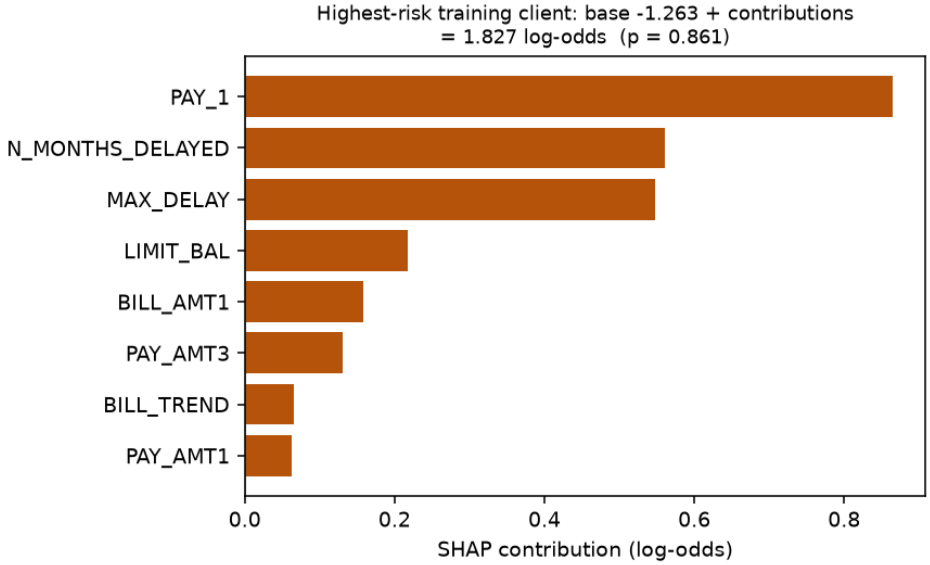


Figure A20: One client decomposed: base value plus each feature's contribution.

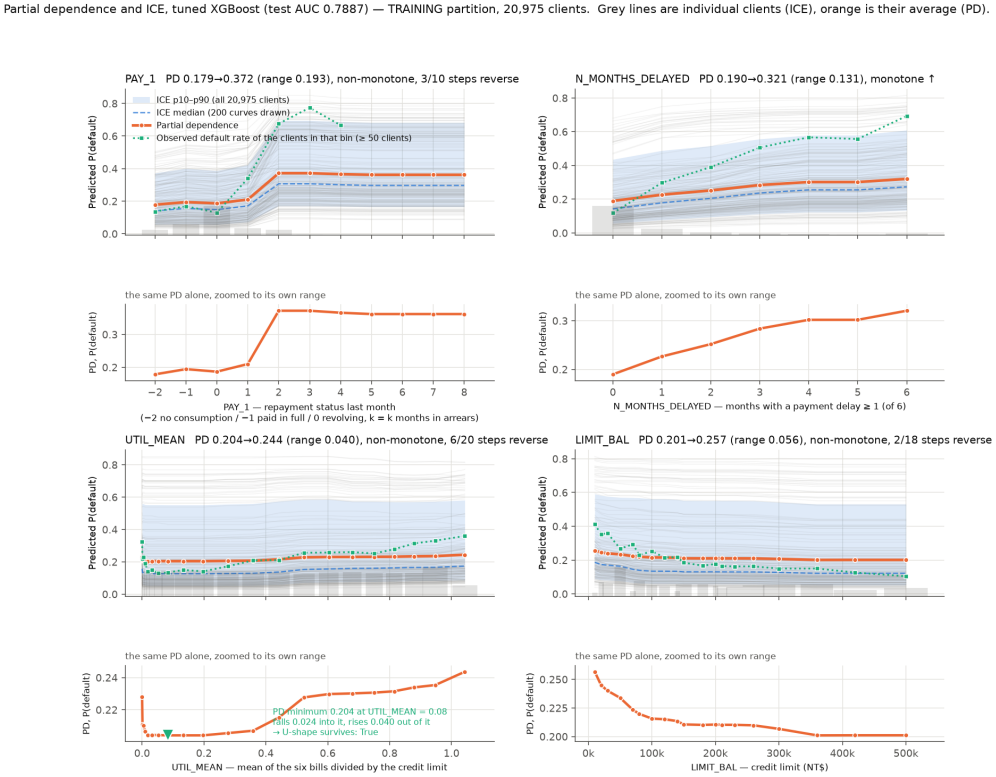


Figure A21: Partial dependence with ICE curves for the four features that matter.

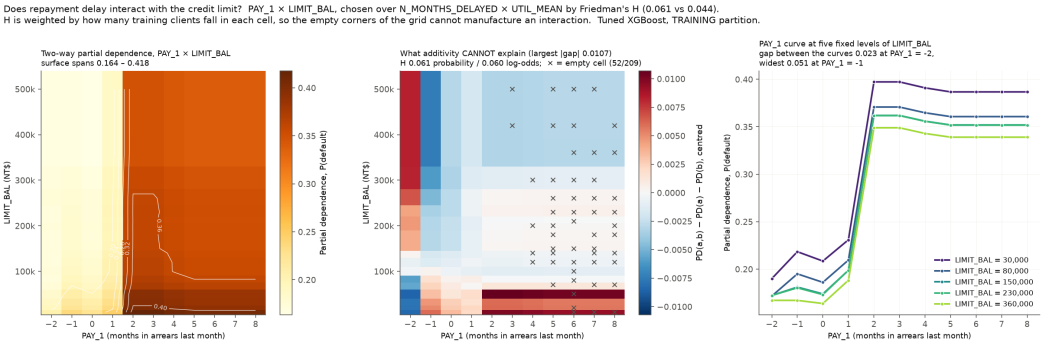


Figure A22: Two-way partial dependence; cells with no training client are crossed.

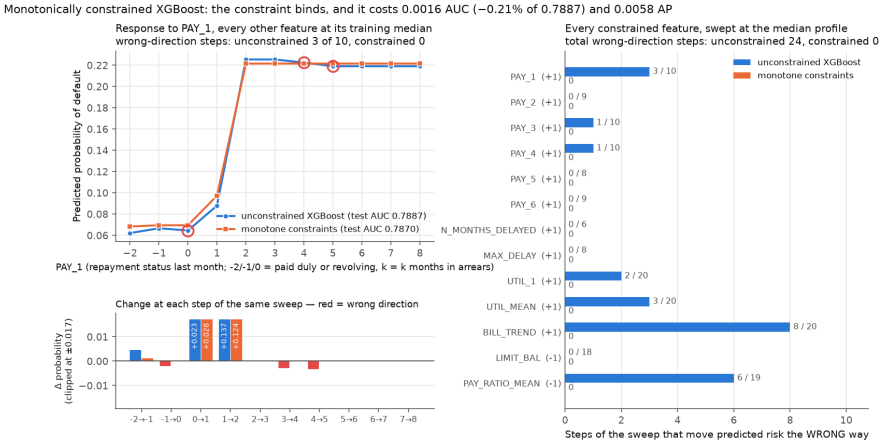


Figure A23: Predicted probability against the most recent repayment status, with and without monotone constraints.

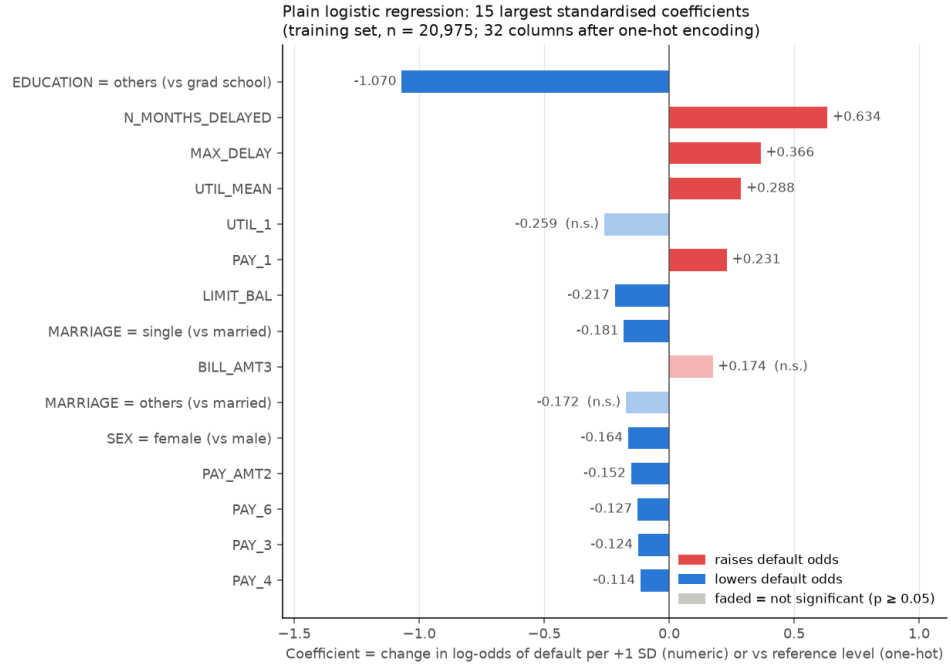


Figure A24: Standardised logistic coefficients.

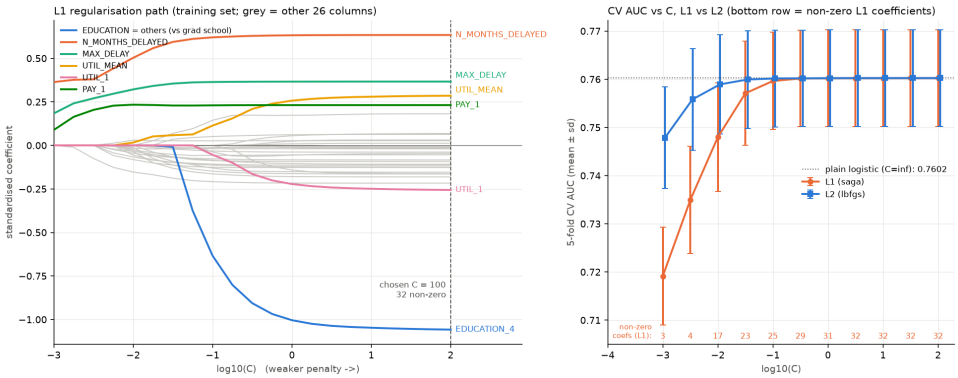


Figure A25: Lasso path: order of entry of the predictors as the penalty is relaxed.

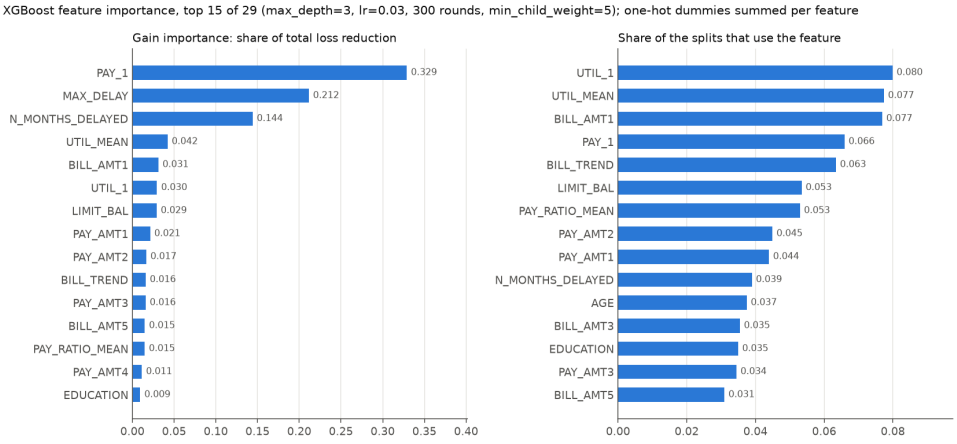


Figure A26: XGBoost gain-based importance.

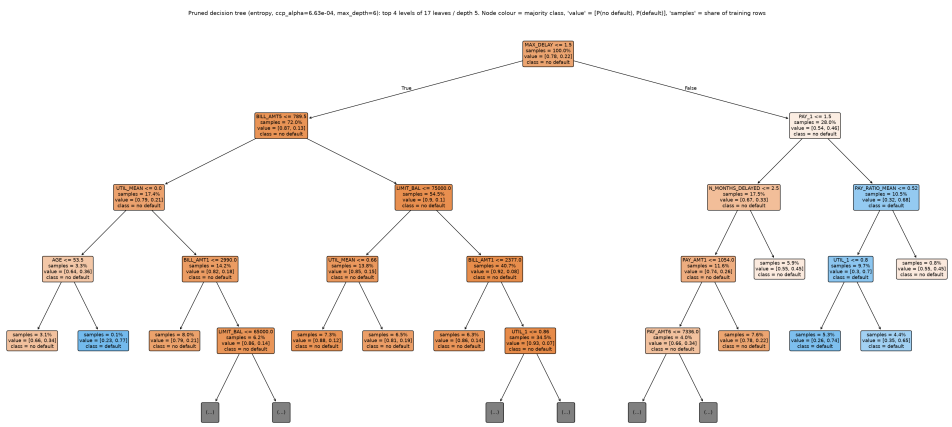


Figure A27: The pruned decision tree, top levels.

A.4 Class imbalance and the leakage demonstration

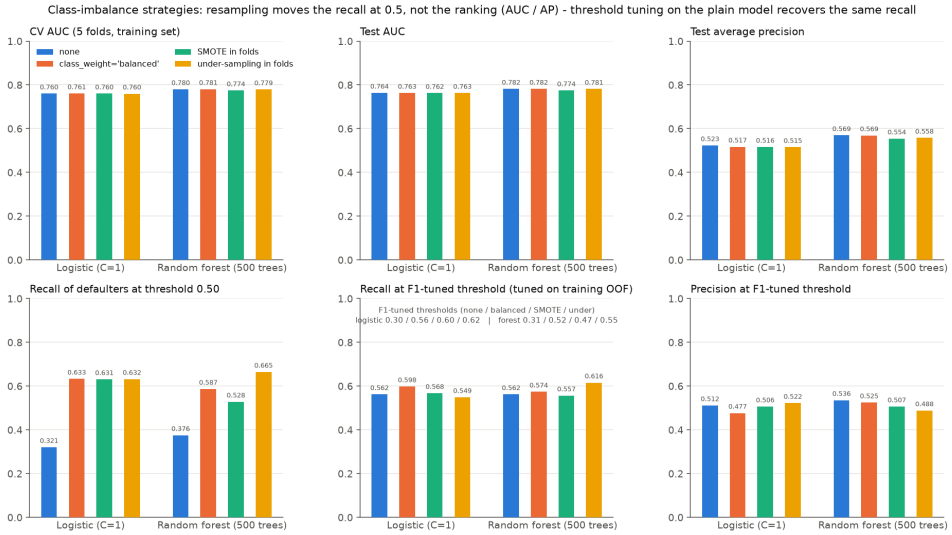


Figure A28: Resampling strategies applied inside the folds: AUC is unchanged, the operating point moves.

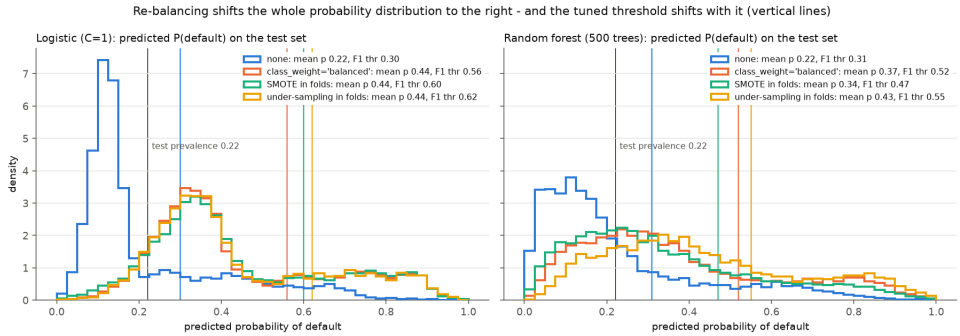


Figure A29: Where each resampling strategy puts the tuned threshold.

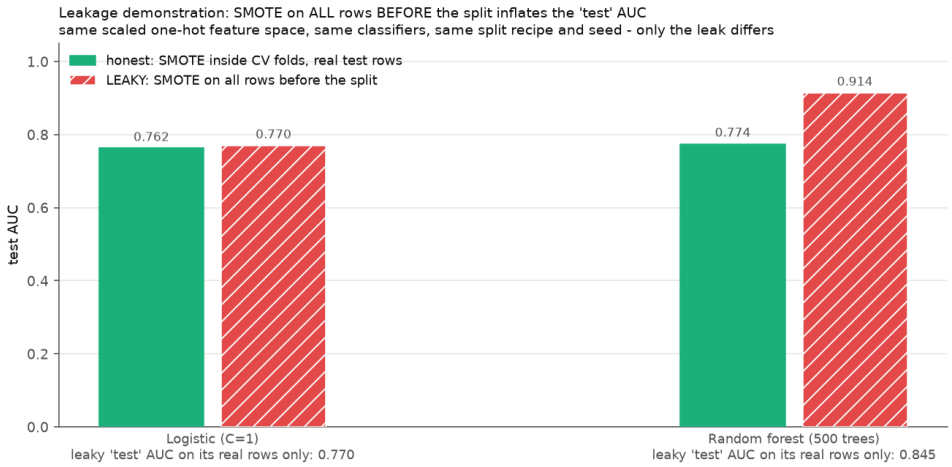


Figure A30: Negative control: SMOTE applied before the split inflates the apparent test AUC.

A.5 Model selection diagnostics

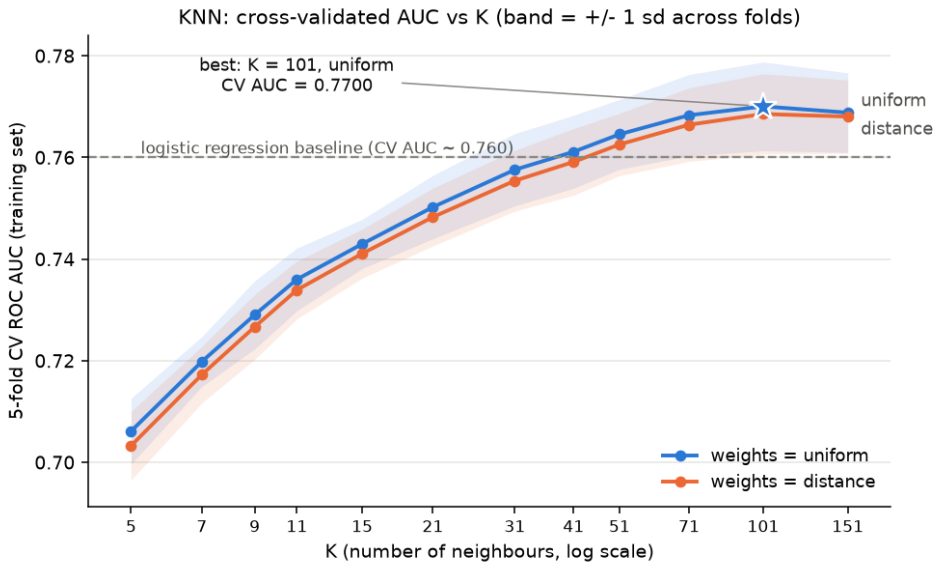


Figure A31: KNN: cross-validated AUC against K.

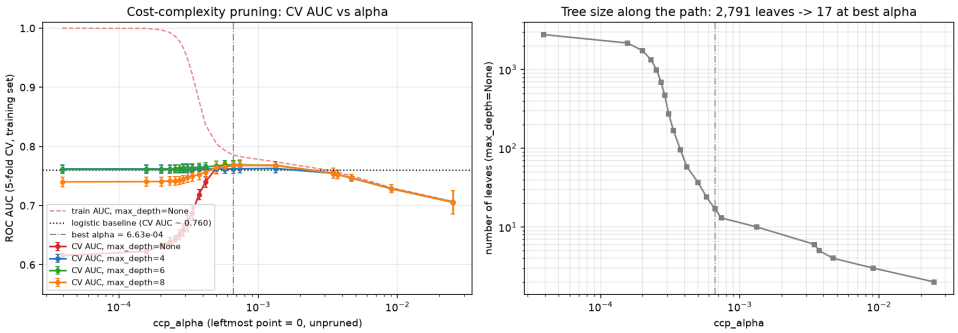


Figure A32: Cost-complexity pruning path.

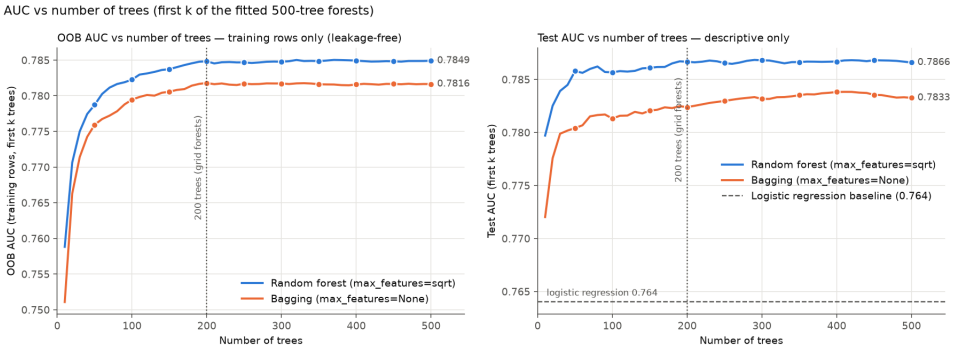


Figure A33: Random forest: out-of-bag and test AUC against the number of trees.

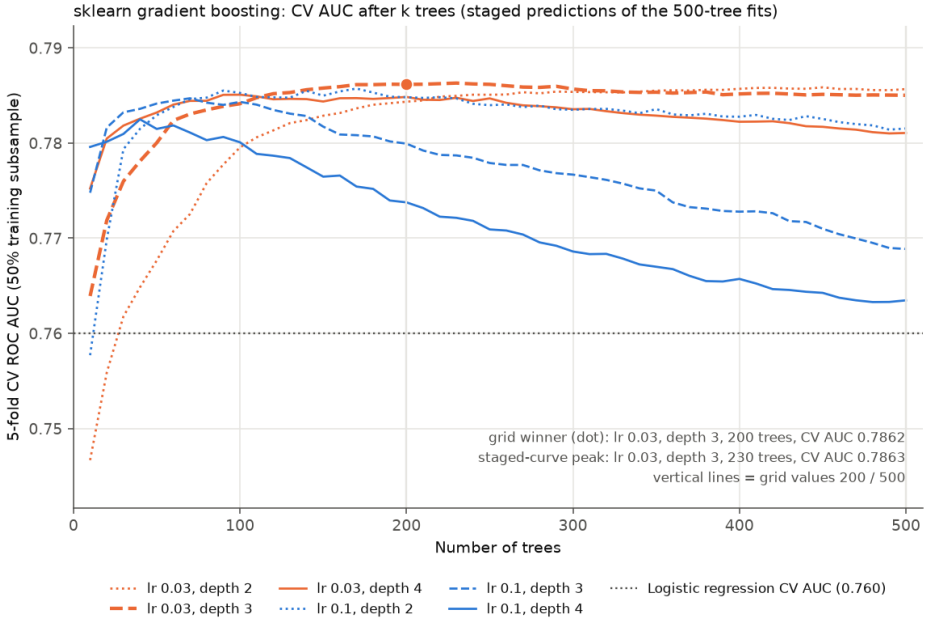


Figure A34: Gradient boosting: cross-validated AUC against the number of rounds.

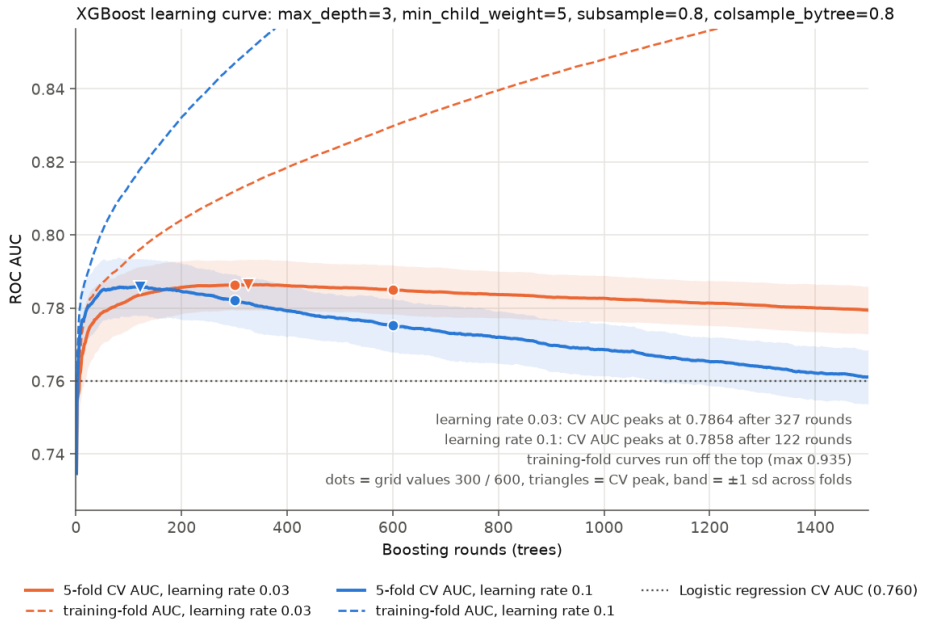


Figure A35: XGBoost learning curves at two learning rates.

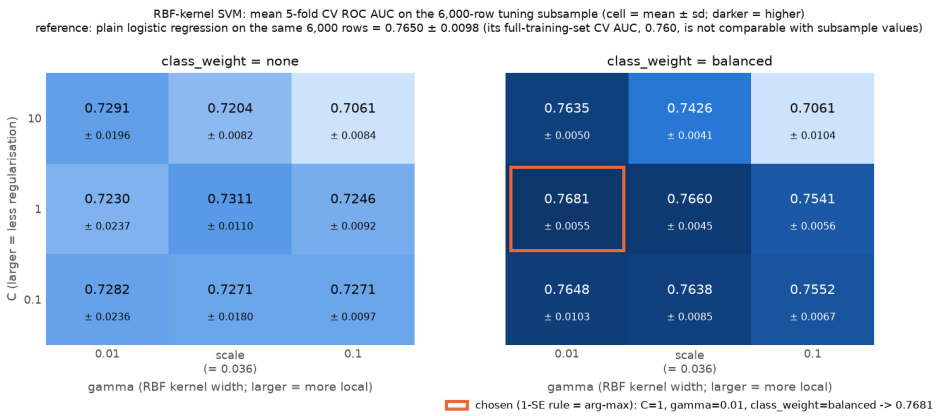


Figure A36: RBF support-vector machine: cross-validated AUC over C and γ.

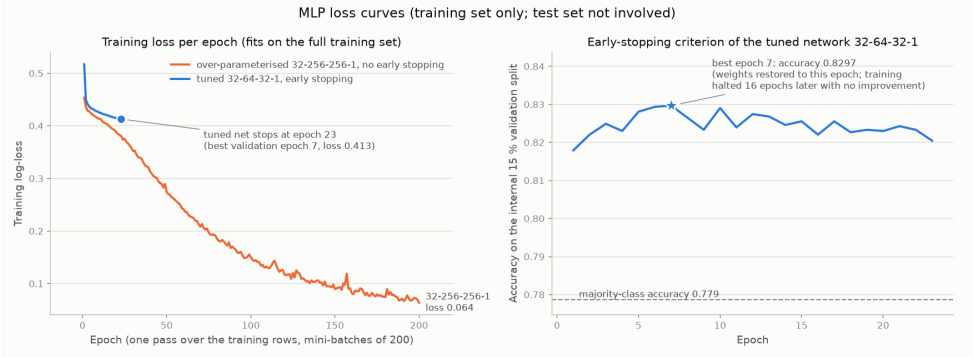


Figure A37: Neural network: training loss and validation AUC by epoch.

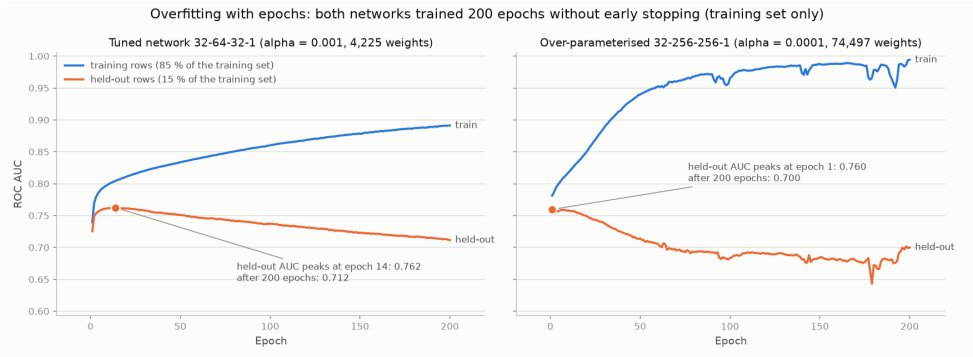


Figure A38: Over-parameterised network: the over-fitting diagnostic.

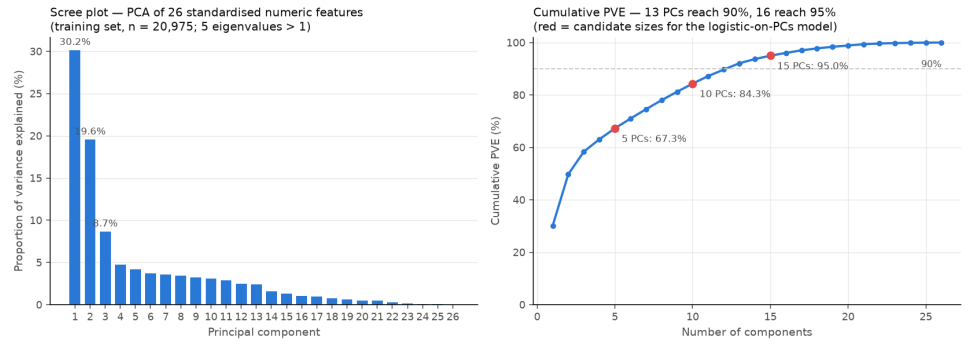


Figure A39: PCA scree plot and cumulative variance explained.

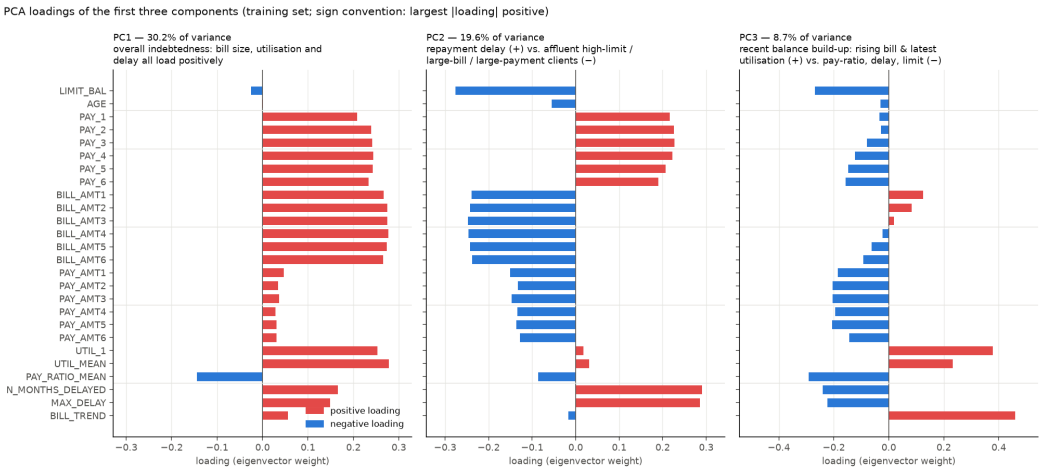


Figure A40: Loadings of the first three principal components.

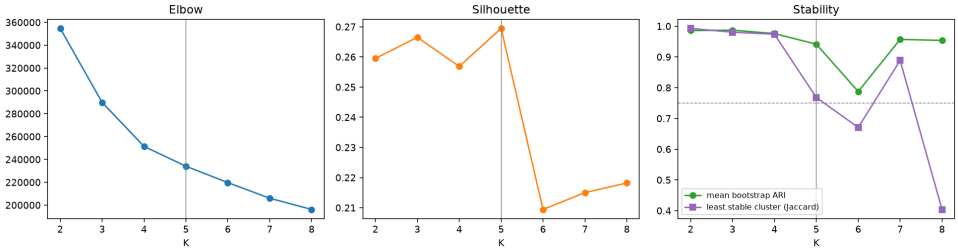


Figure A41: K-means on the first 10 principal components: silhouette, bootstrap ARI and minimum cluster Jaccard against K (Table A16); reproduction on 21 October of RUAN Hangjing's 30 September analysis, training partition only.